DS4023（2024 Fall）
Machine Learning
Department of Computer Science
BNU-HKBU United International College

Chapter 4: UnSupervised Learning

Outline

- Gaussian mixture models
- EM algorithm
- Mean-shift algorithm
- KDE
- Self organizing map

Gaussian mixture models and the EM algorithm

Chapter 4: UnSupervised Learning

Unsupervised Learning

- You walk into a bar.

  A stranger approaches and tells you:

  > "I've got data from k classes. Each class produces observations with a normal distribution and variance $\sigma^2 I$. Standard simple multivariate Gaussian assumptions. I can tell you all the $P(w_i)$'s ."

- So far, looks straightforward.

  > "I need a maximum likelihood estimate of the $\boldsymbol{\mu}_i$'s ."

- No problem:

  > "There's just one thing. None of the data are labeled. I have datapoints, but I don't know what class they're from (any of them!)

- Uh oh!!

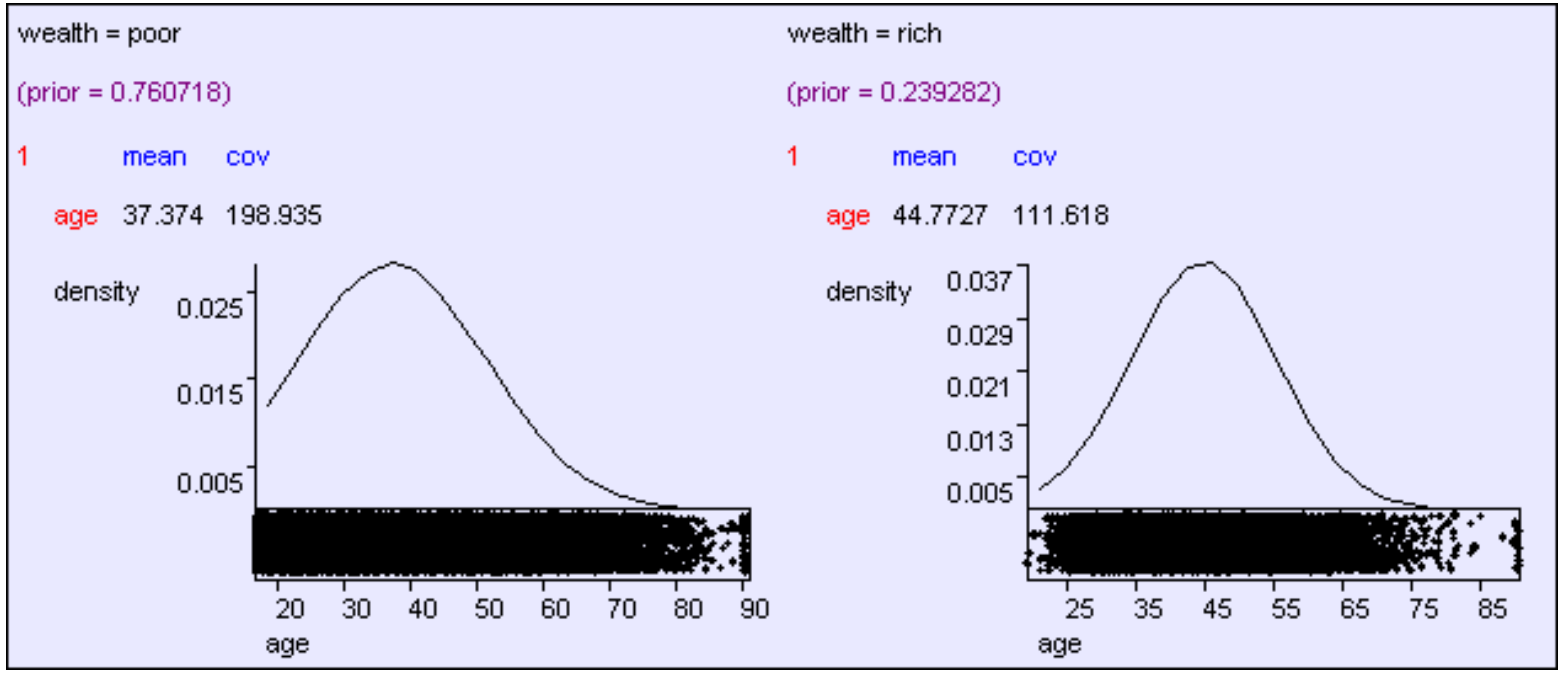
Bayes Classifier

$$P(y=i \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid y=i)P(y=i)}{p(\mathbf{x})}$$

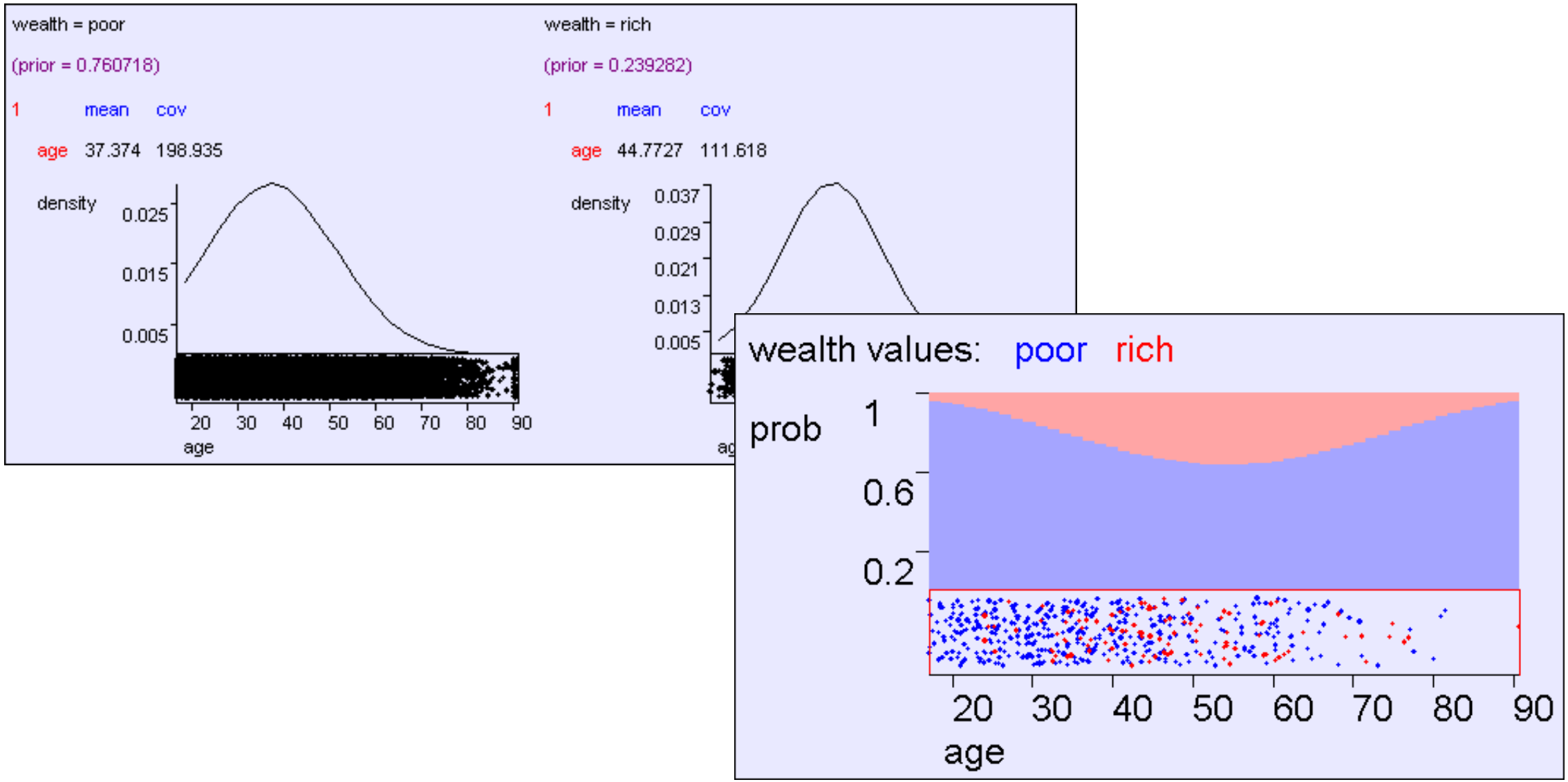
$$P(y=i \mid \mathbf{x}) = \frac{\frac{1}{(2\pi)^{m/2} \parallel \boldsymbol{\Sigma}_i \parallel^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x}_k - \boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i (\mathbf{x}_k - \boldsymbol{\mu}_i)\right] p_i}{p(\mathbf{x})}$$

How do we deal with that?

Predicting wealth from age

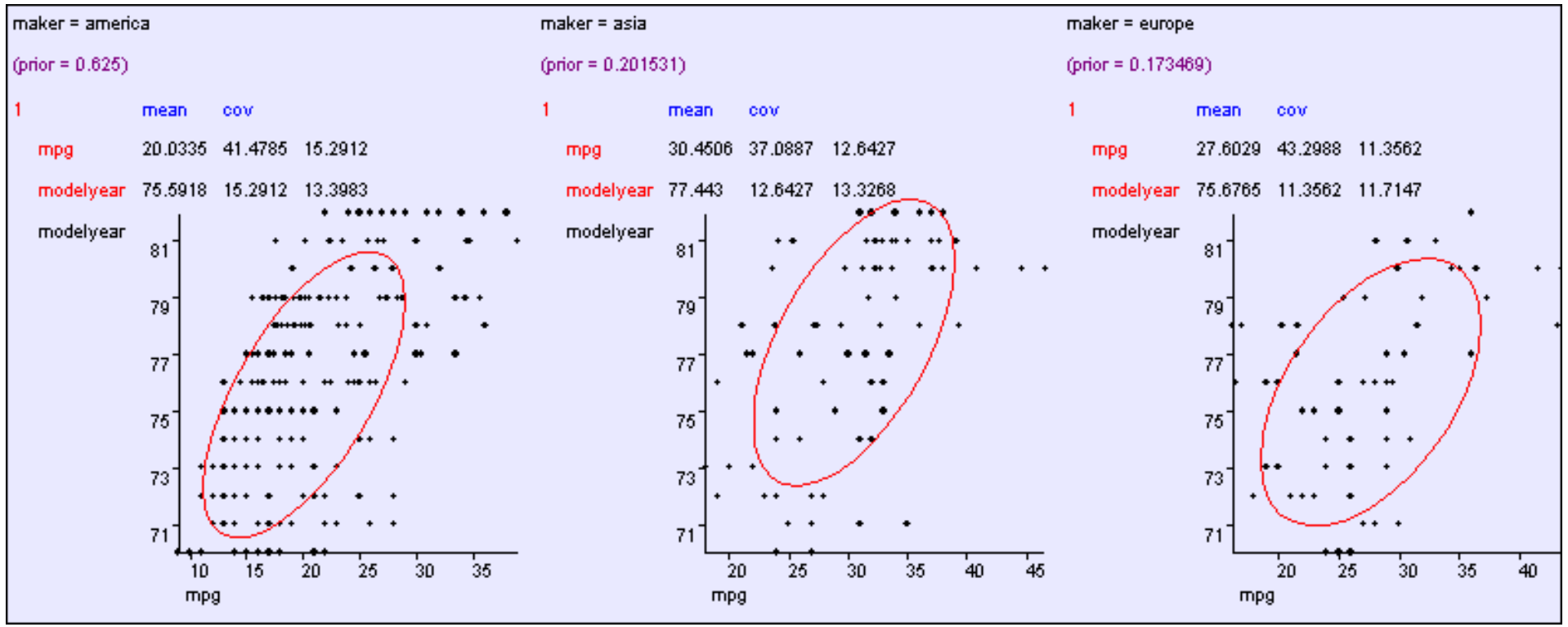


Predicting wealth from age



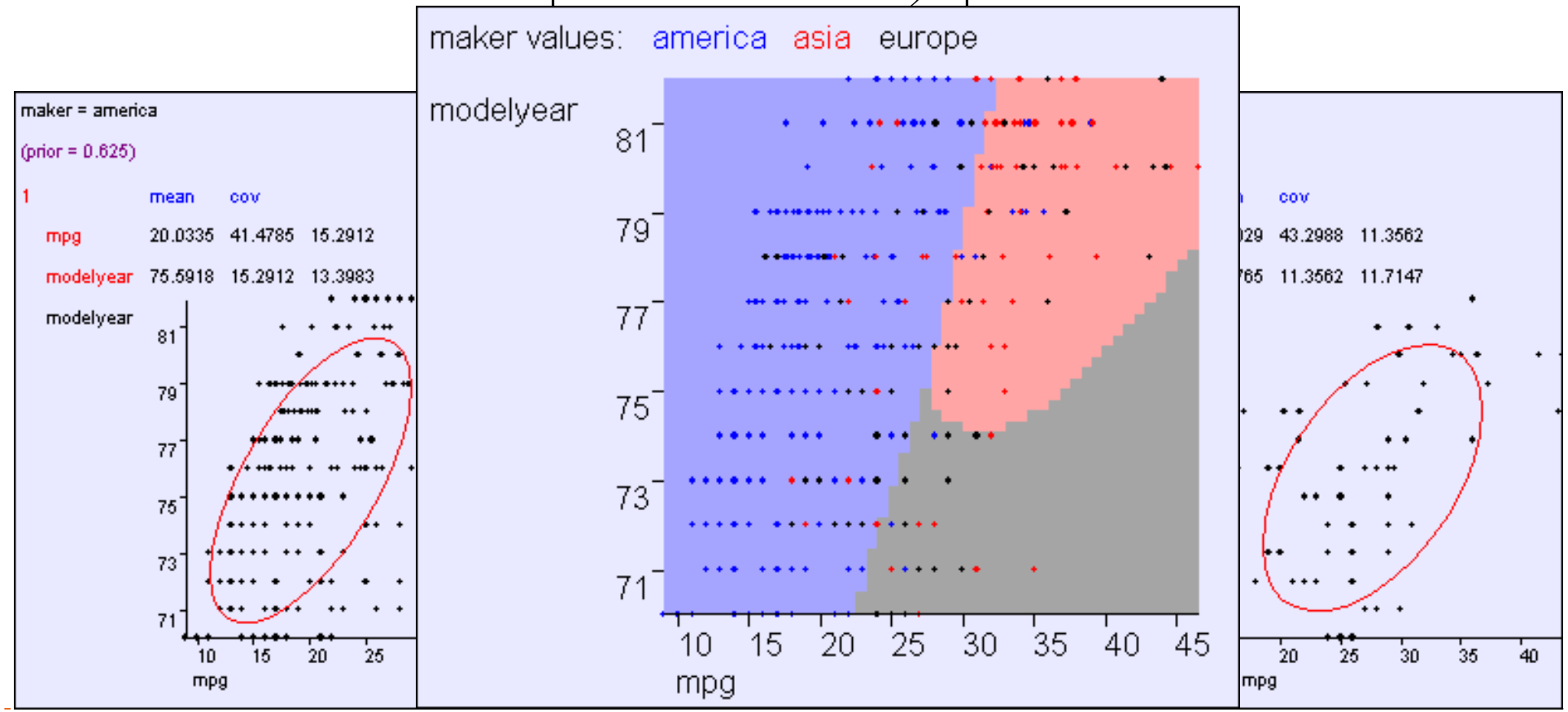
Learning model, year, mpg ---> maker

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma^2_1 & \sigma_{12} & \cdots & \sigma_{1m} \\ \sigma_{12} & \sigma^2_2 & \cdots & \sigma_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{1m} & \sigma_{2m} & \cdots & \sigma^2_m \end{pmatrix}$$



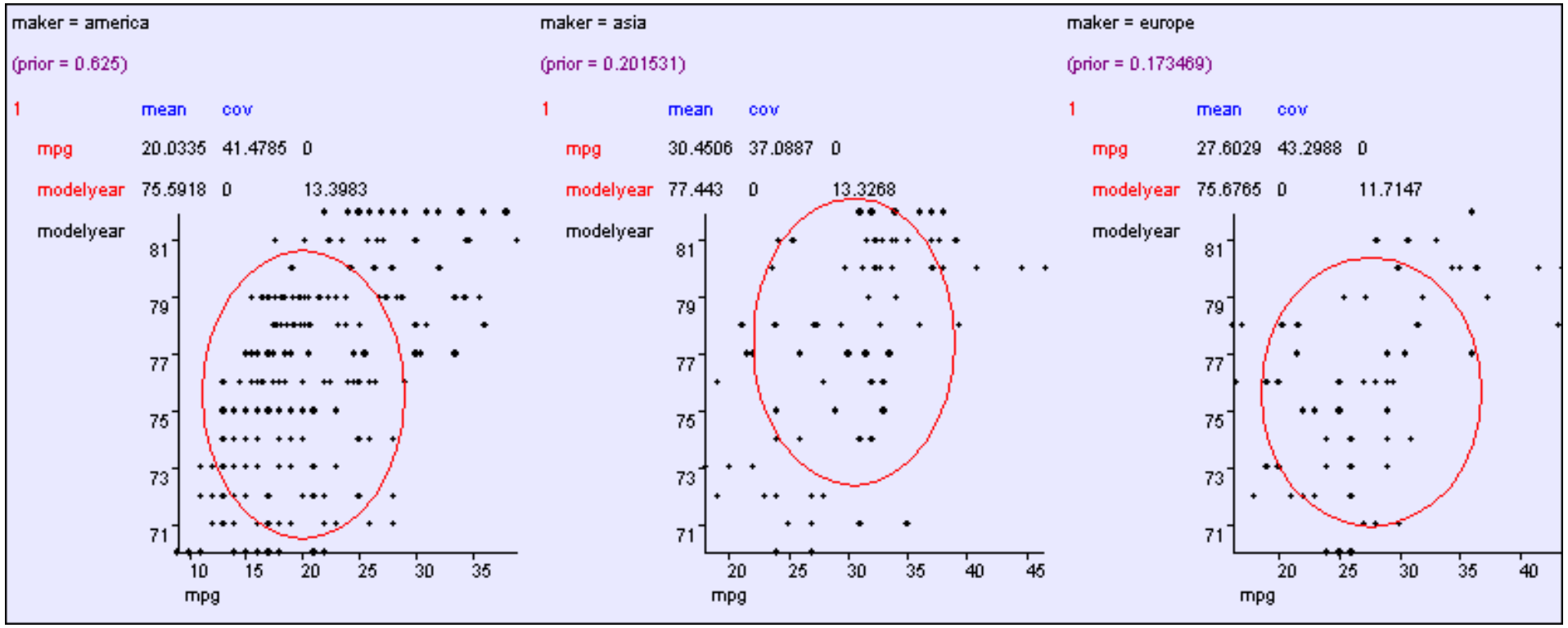
General: $O(m^2)$ parameters

$$\Sigma = \begin{pmatrix} \sigma^2_1 & \sigma_{12} & \cdots & \sigma_{1m} \\ \sigma_{12} & \sigma^2_2 & \cdots & \sigma_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ & & & \sigma^2 \end{pmatrix}$$



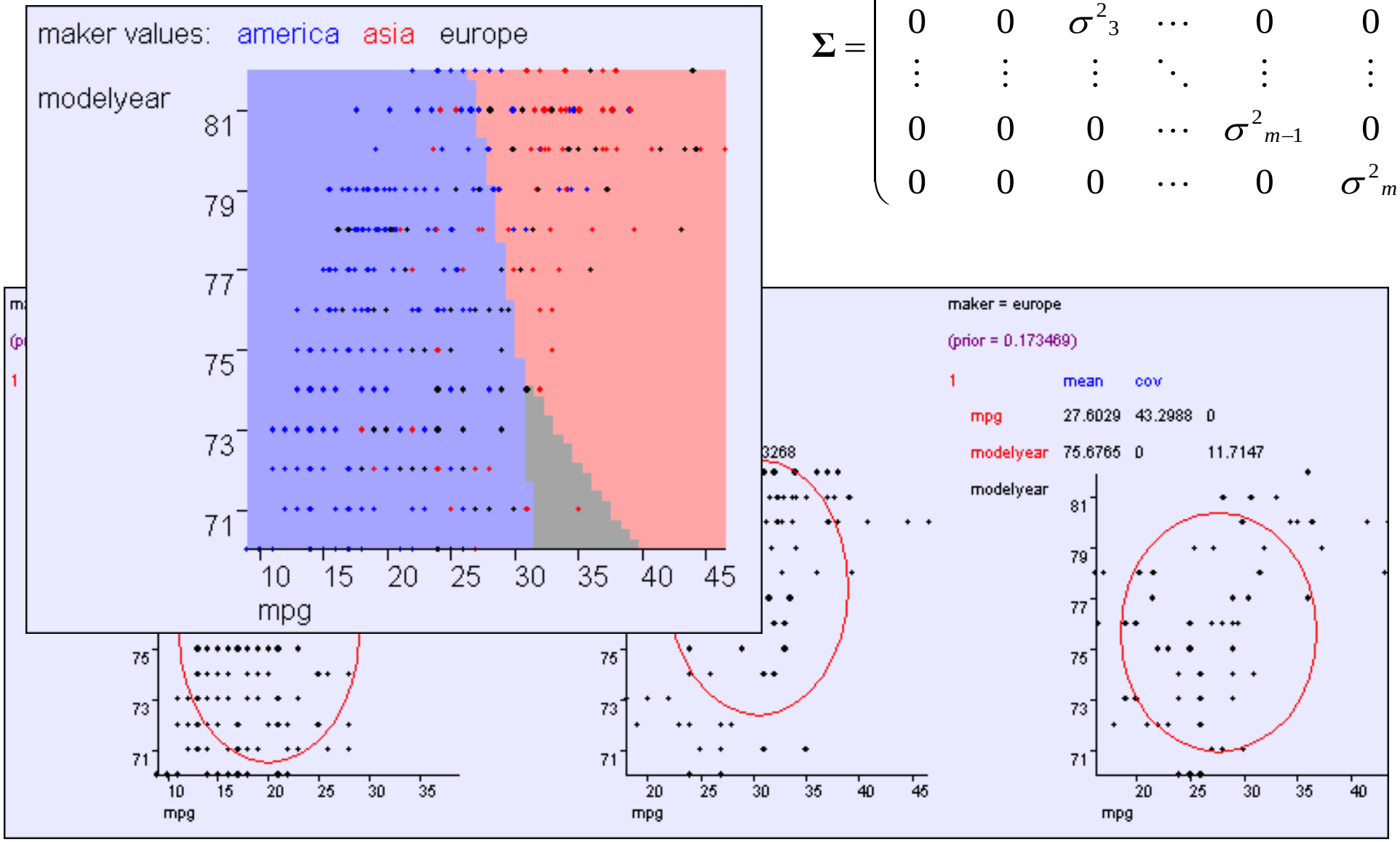
Aligned: *O(m)* parameters

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma^2_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & \sigma^2_2 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \sigma^2_3 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & \sigma^2_{m-1} & 0 \\ 0 & 0 & 0 & \cdots & 0 & \sigma^2_m \end{pmatrix}$$



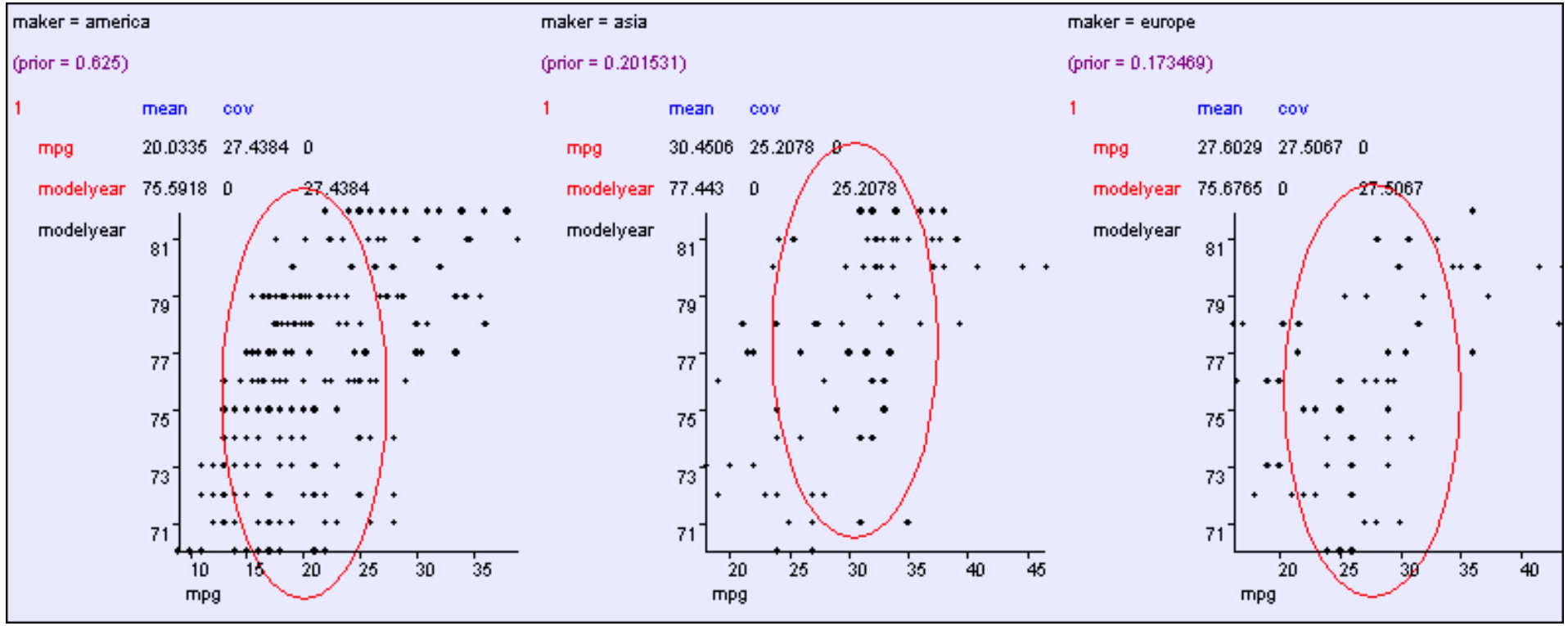
Aligned: *O(m)* parameters

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma^2_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & \sigma^2_2 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \sigma^2_3 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & \sigma^2_{m-1} & 0 \\ 0 & 0 & 0 & \cdots & 0 & \sigma^2_m \end{pmatrix}$$



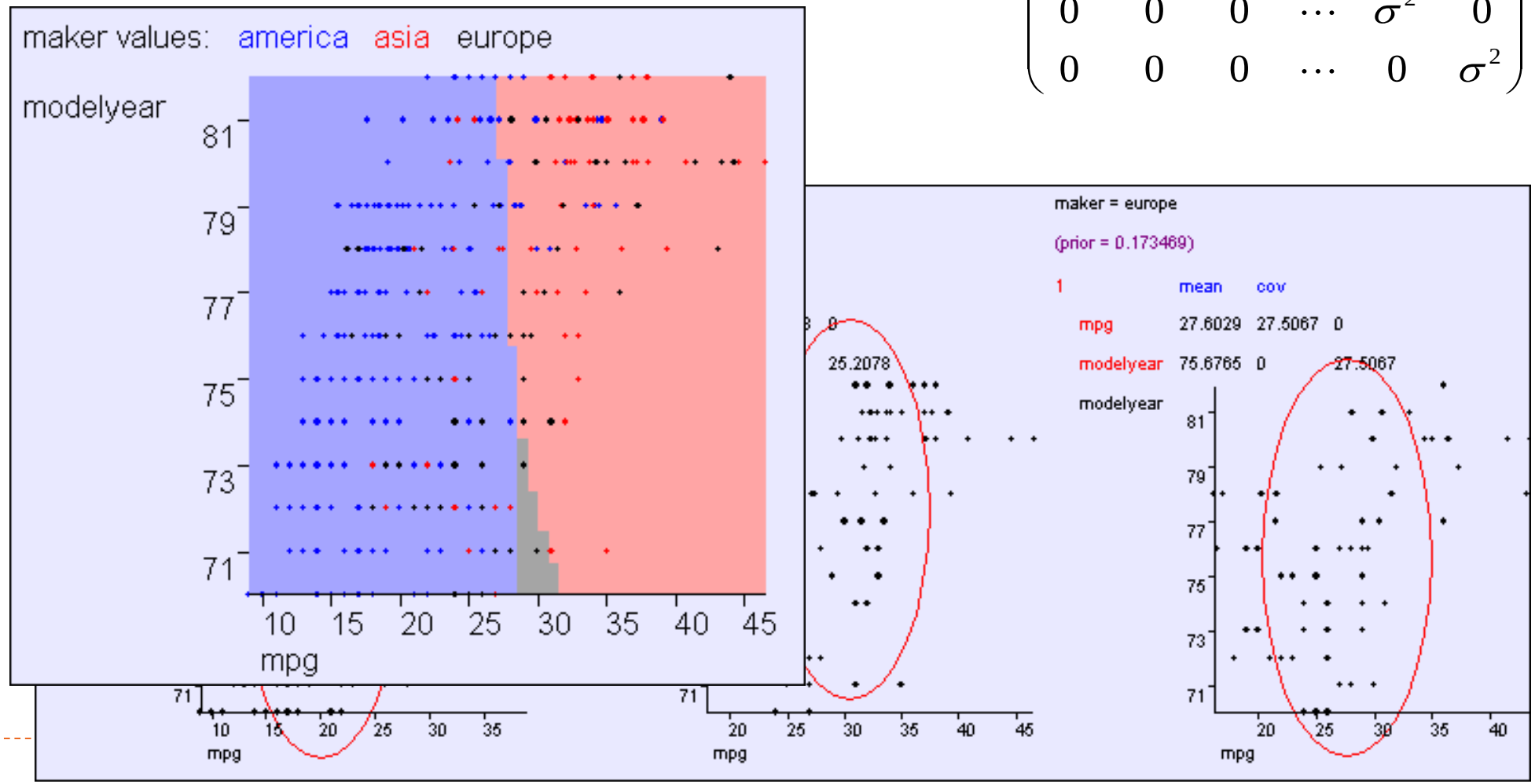
Spherical: *O(1)* cov parameters

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma^2 & 0 & 0 & \cdots & 0 & 0 \\ 0 & \sigma^2 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \sigma^2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & \sigma^2 & 0 \\ 0 & 0 & 0 & \cdots & 0 & \sigma^2 \end{pmatrix}$$

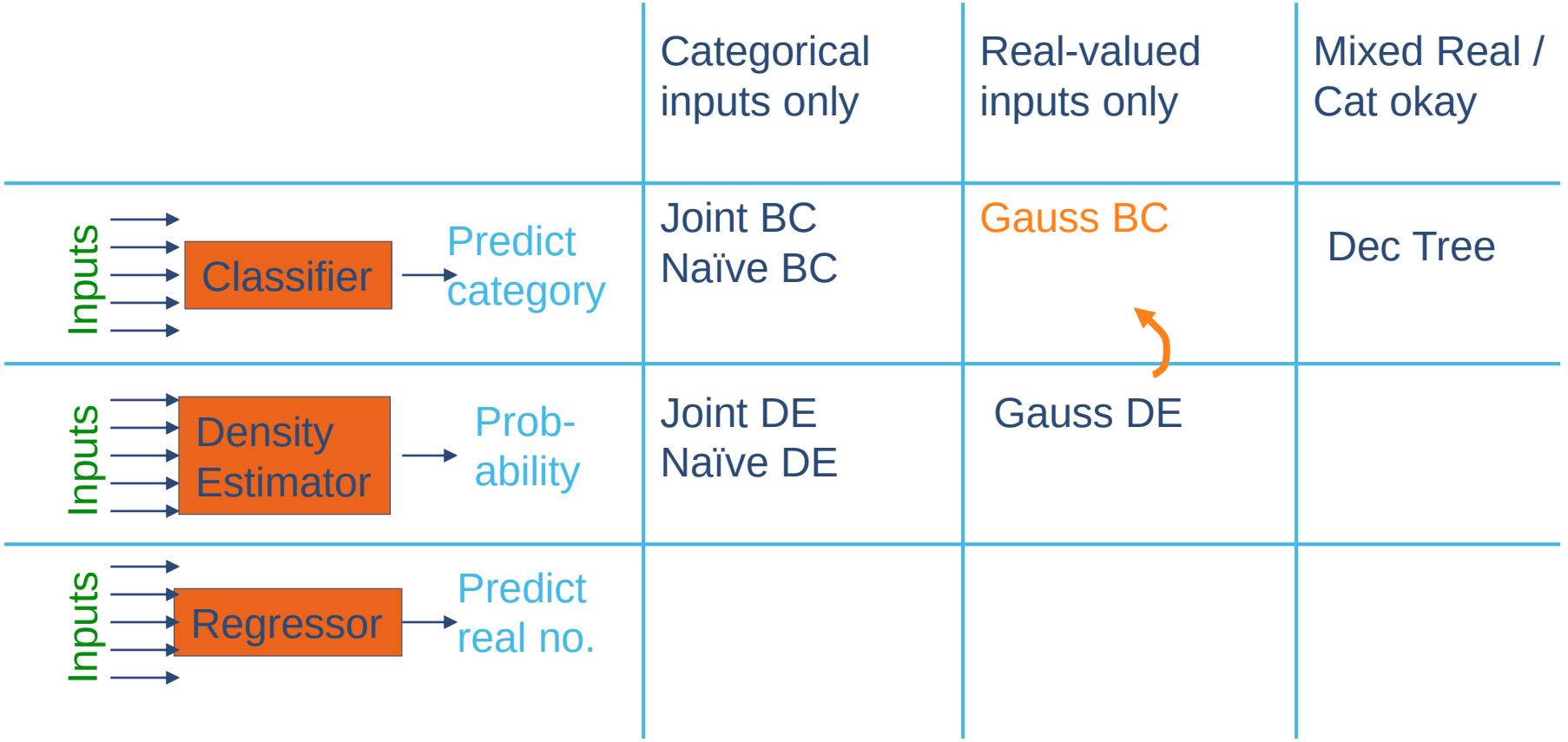


Spherical: *O(1)* cov parameters

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma^2 & 0 & 0 & \cdots & 0 & 0 \\ 0 & \sigma^2 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \sigma^2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & \sigma^2 & 0 \\ 0 & 0 & 0 & \cdots & 0 & \sigma^2 \end{pmatrix}$$

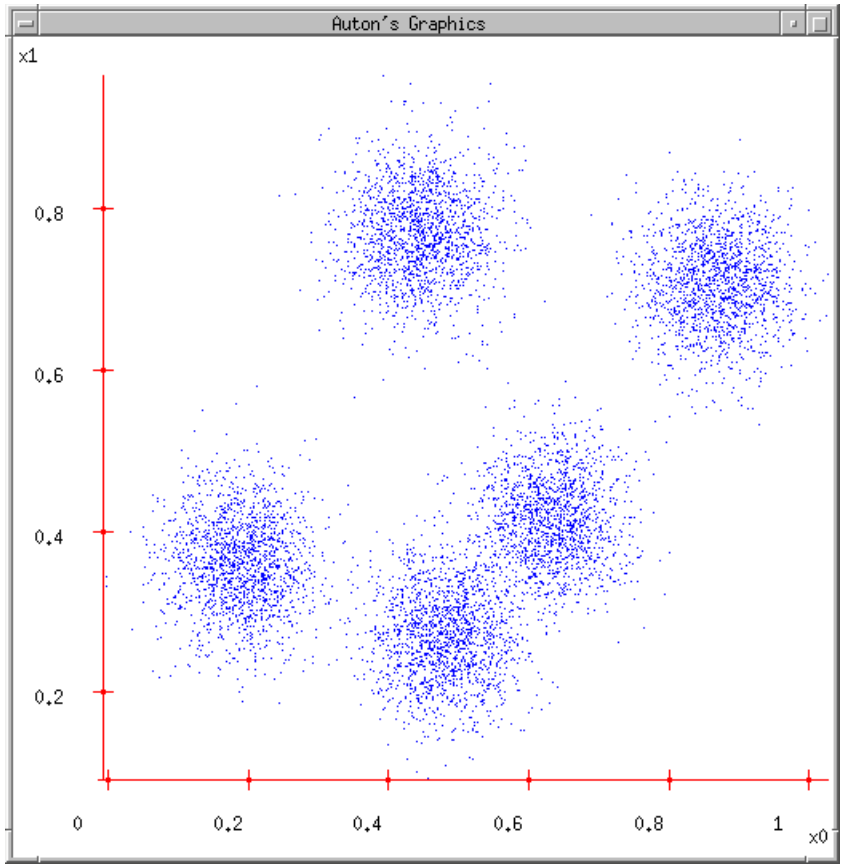


Making a Classifier from a Density Estimator



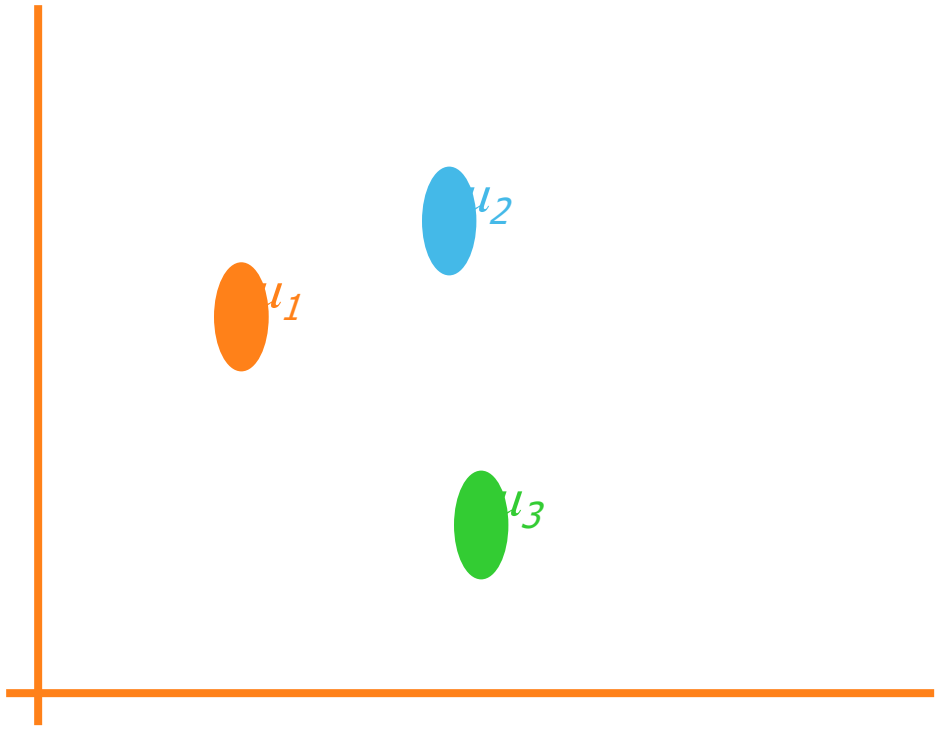
Density Estimation

What if we want to do density estimation with multimodal or clumpy data?



The GMM assumption

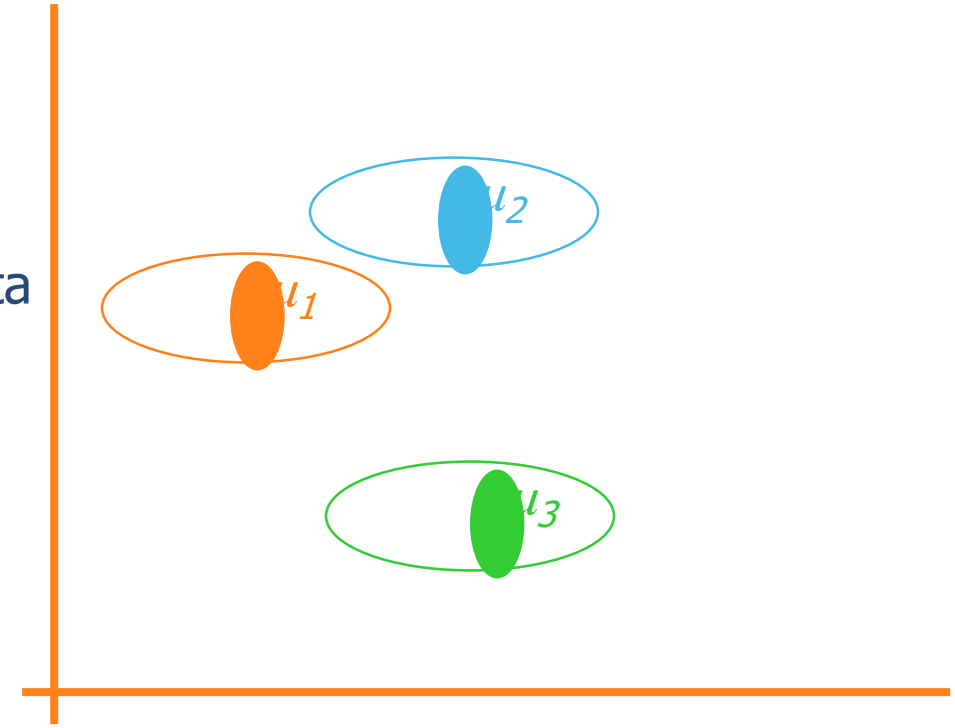
- There are k components. The i'th component is called ω_i
- Component ω_i has an associated mean vector μ_i



The GMM assumption

- There are k components. The i'th component is called ω_i
- Component ω_i has an associated mean vector μ_i
- Each component generates data from a Gaussian with mean μ_i and covariance matrix $\sigma^2\boldsymbol{I}$

Assume that each datapoint is generated according to the following recipe:

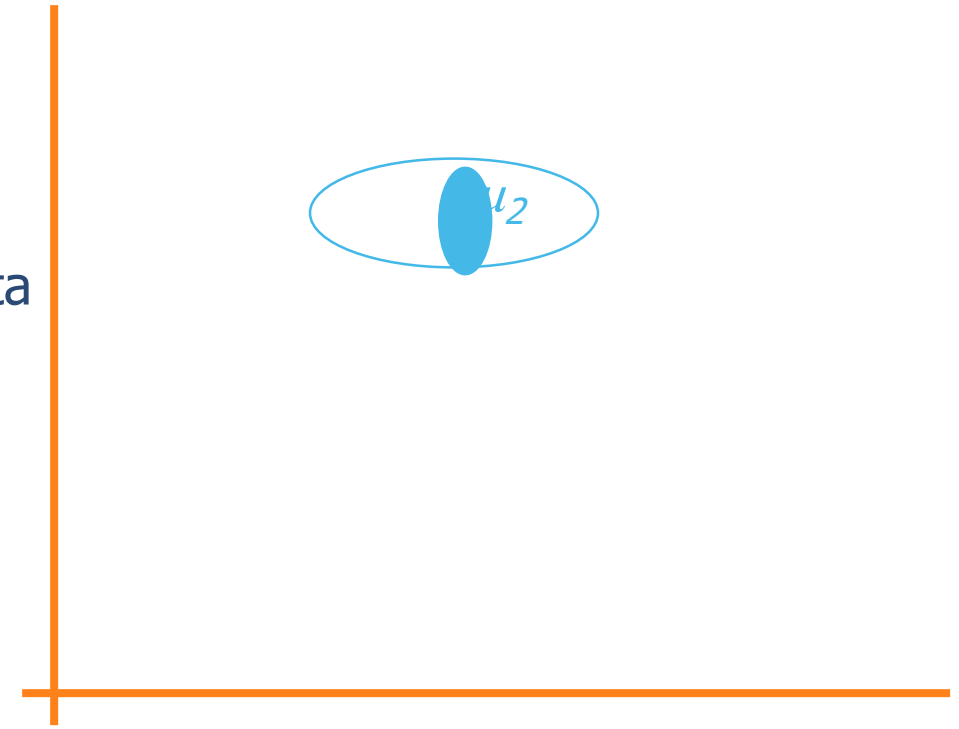


The GMM assumption

- There are k components. The i'th component is called ω_i
- Component ω_i has an associated mean vector μ_i
- Each component generates data from a Gaussian with mean μ_i and covariance matrix $\sigma^2\boldsymbol{I}$

Assume that each datapoint is generated according to the following recipe:

1. Pick a component at random. Choose component i with probability $P(\omega_i)$.

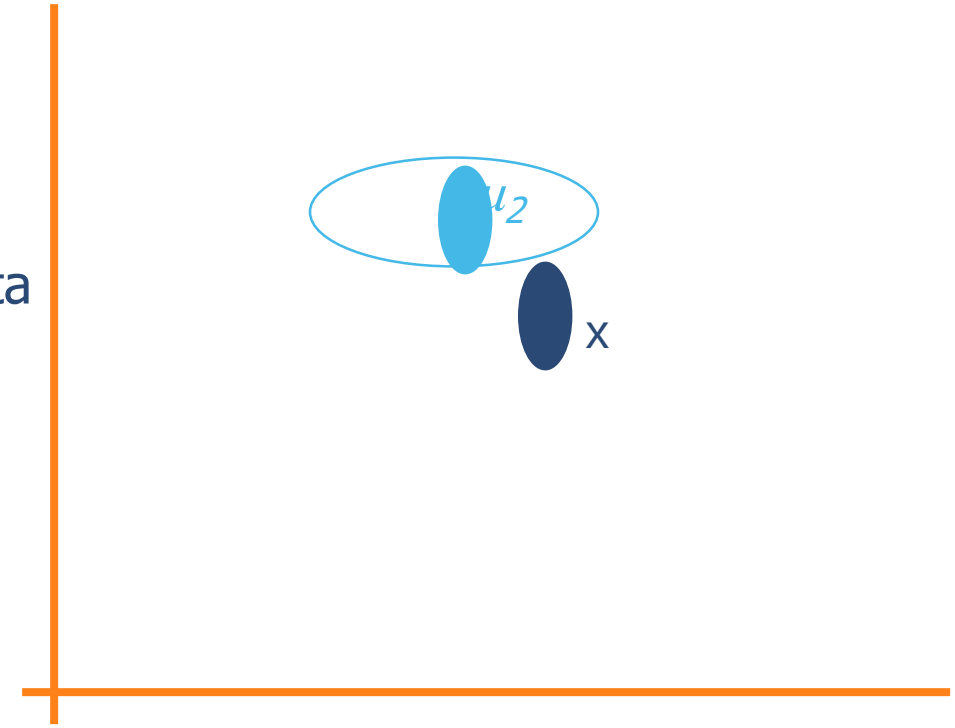


The GMM assumption

- There are k components. The i'th component is called ω_i
- Component ω_i has an associated mean vector μ_i
- Each component generates data from a Gaussian with mean μ_i and covariance matrix $\sigma^2\boldsymbol{I}$

Assume that each datapoint is generated according to the following recipe:

1. Pick a component at random. Choose component i with probability $P(\omega_i)$.
2. Datapoint ~ N(μ_i, $\sigma^2\boldsymbol{I}$)

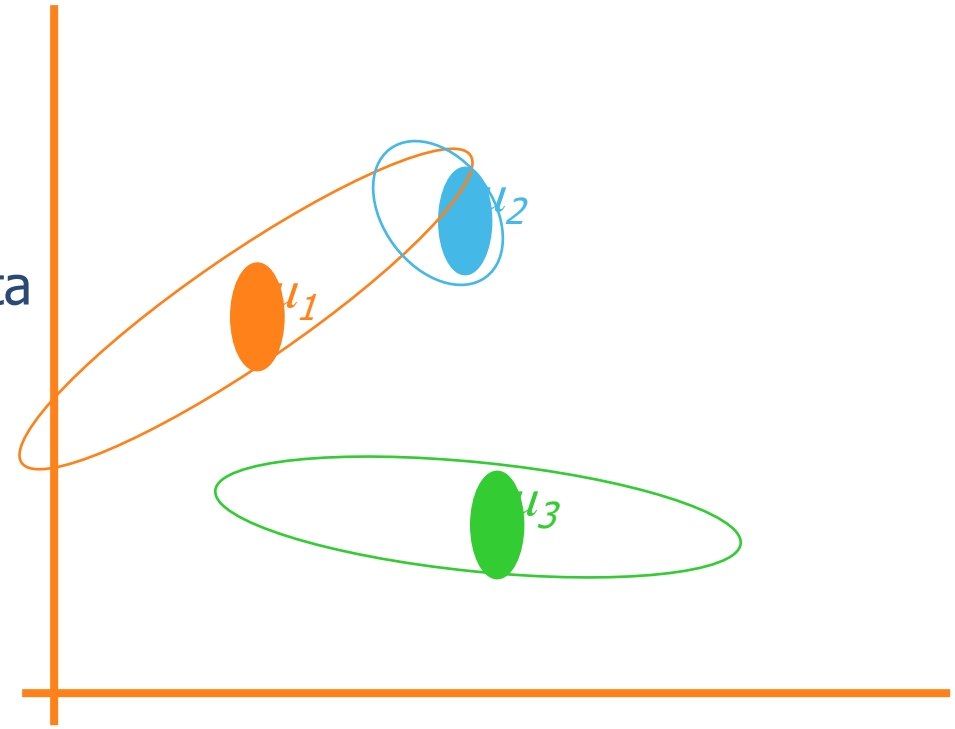


The General GMM assumption

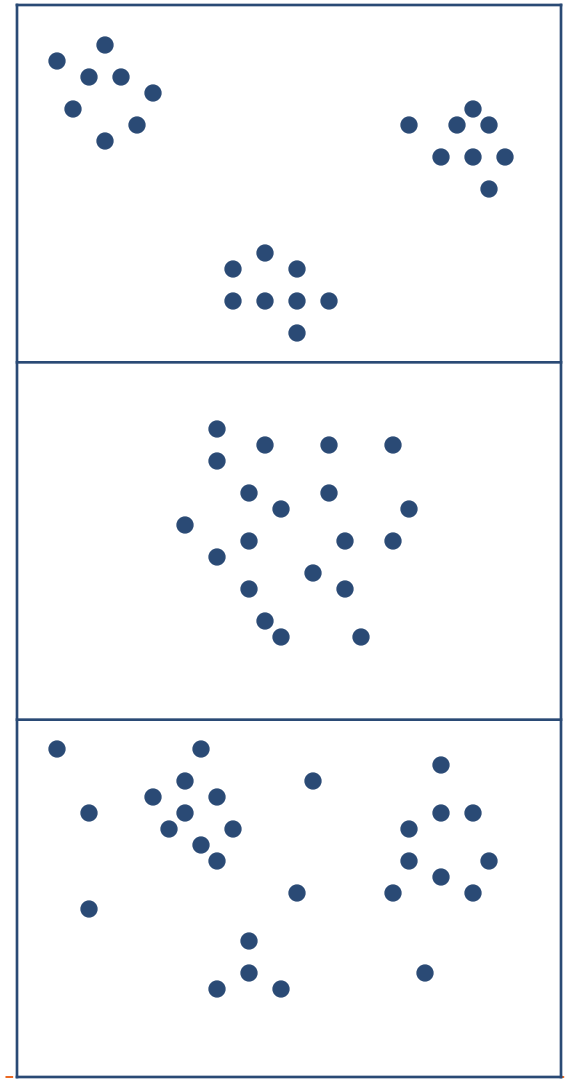
- There are k components. The i'th component is called ω_i
- Component ω_i has an associated mean vector μ_i
- Each component generates data from a Gaussian with mean μ_i and covariance matrix Σ_i

Assume that each datapoint is generated according to the following recipe:

1. Pick a component at random. Choose component i with probability $P(\omega_i)$.
2. Datapoint ~ N(μ_i, Σ_i)



Unsupervised Learning: not as hard as it looks



Sometimes easy

Sometimes impossible

and sometimes
in between

*IN CASE YOU'RE WONDERING WHAT THESE DIAGRAMS ARE, THEY SHOW 2-d UNLABELED DATA (**X** VECTORS) DISTRIBUTED IN 2-d SPACE. THE TOP ONE HAS THREE VERY CLEAR GAUSSIAN CENTERS*

Computing likelihoods in unsupervised case

We have $\boldsymbol{x}_1$, $\boldsymbol{x}_2$, … $\boldsymbol{x}_N$

We know P(w_1) P(w_2) .. P(w_k)

We know σ

P($\boldsymbol{x}$|w_i, $\boldsymbol{\mu}_i$, … $\boldsymbol{\mu}_k$) = Prob that an observation from class w_i would have value $\boldsymbol{x}$ given class means $\boldsymbol{\mu}_1$… $\boldsymbol{\mu}_x$

Can we write an expression for that?

likelihoods in unsupervised case

We have $\boldsymbol{x}_1$ $\boldsymbol{x}_2$ … $\boldsymbol{x}_n$
We have P(w_1) .. P(w_k). We have σ.
We can define, for any $\boldsymbol{x}$, P($\boldsymbol{x}$|w_i , $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_2$ .. $\boldsymbol{\mu}_k$)

Can we define P($\boldsymbol{x}$ | $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_2$ .. $\boldsymbol{\mu}_k$) ?

Can we define P($\boldsymbol{x}_1$, $\boldsymbol{x}_1$, .. $\boldsymbol{x}_n$ | $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_2$ .. $\boldsymbol{\mu}_k$) ?

[YES, IF WE ASSUME THE X_1'S WERE DRAWN INDEPENDENTLY]

We now have a procedure s.t. if you give me a guess at $\mu_1, \mu_2 .. \mu_k$,

I can tell you the prob of the unlabeled data given those μ's.

Suppose x's are 1-dimensional.

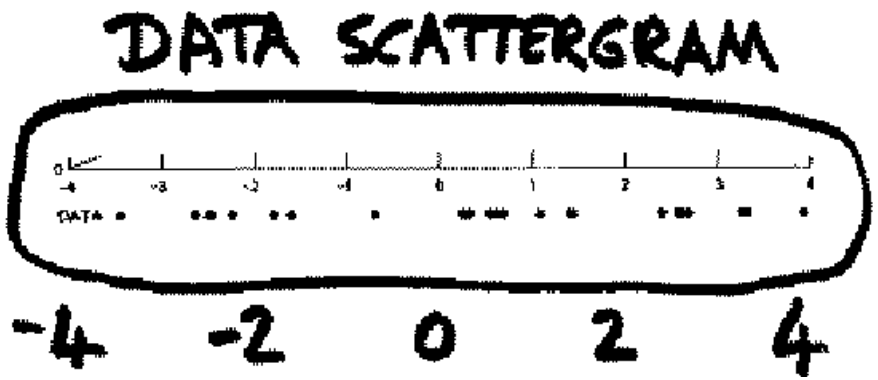
There are two classes; w_1 and w_2

$P(w_1) = 1/3$ $P(w_2) = 2/3$ $\sigma = 1$.

There are 25 unlabeled datapoints

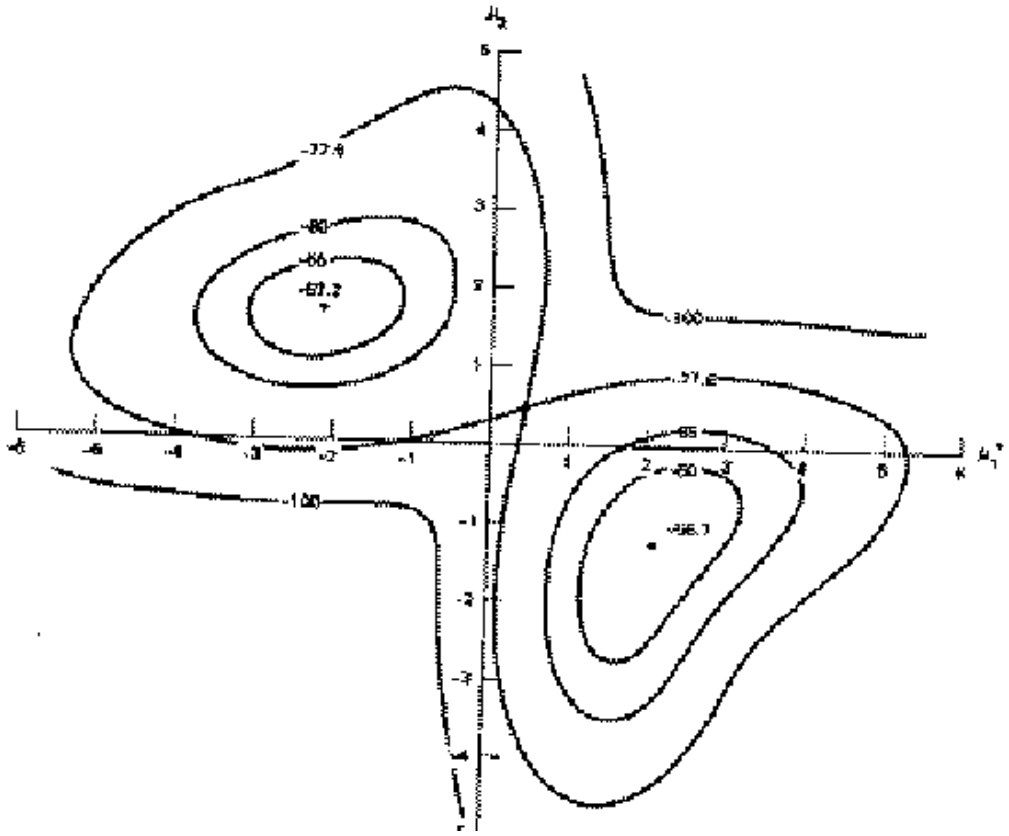
$x_1 = 0.608$
$x_2 = -1.590$
$x_3 = 0.235$
$x_4 = 3.949$
:
$x_{25} = -0.712$

(From Duda and Hart)



Duda & Hart's Example

Graph of
$\log P(x_1, x_2 .. x_{25} \mid \mu_1, \mu_2)$
against μ_1 ($\rightarrow$) and μ_2 ($\uparrow$)



Max likelihood = (μ_1 =-2.13, μ_2 =1.668)

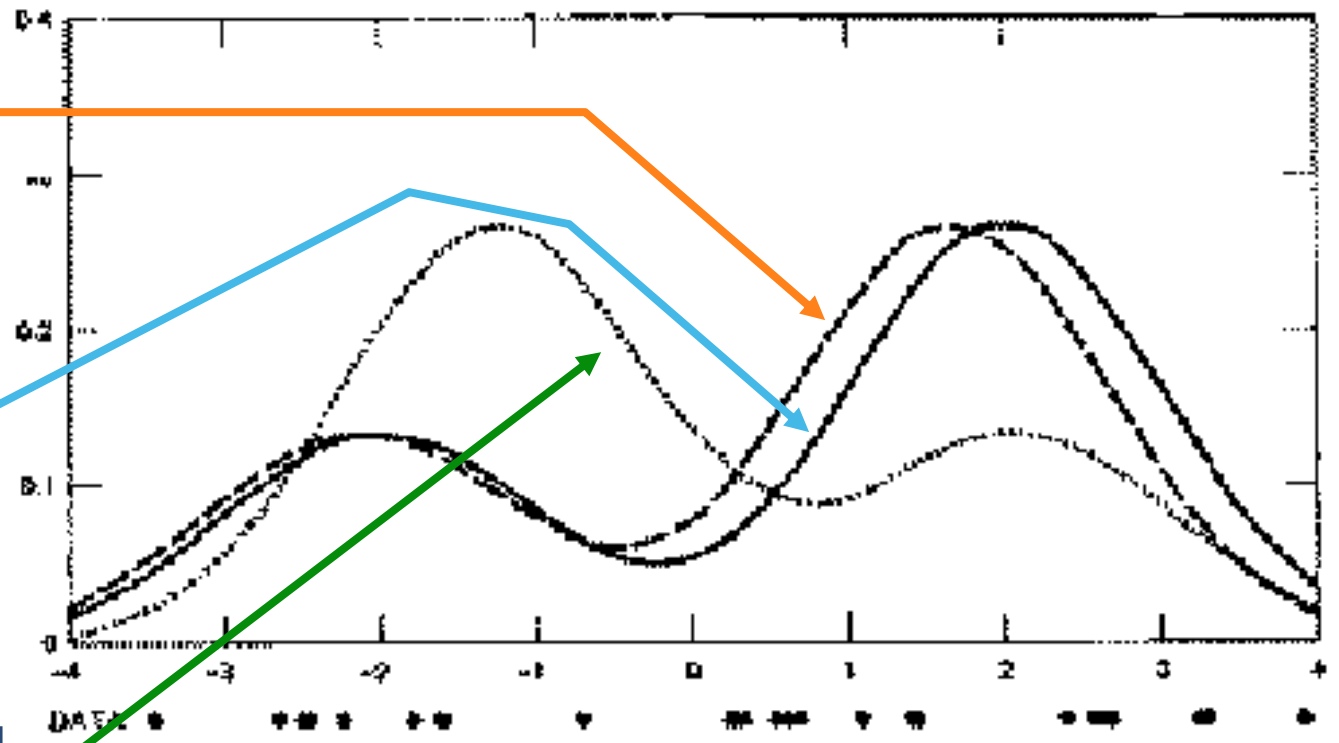
Local minimum, but very close to global at (μ_1 =2.085, μ_2 =-1.257)*

* corresponds to switching w_1 + w_2.

Duda & Hart's Example

We can graph the prob. dist. function of data given our μ_1 and μ_2 estimates.

We can also graph the true function from which the data was randomly generated.



- They are close. Good.
- The 2nd solution tries to put the "2/3" hump where the "1/3" hump should go, and vice versa.
- In this example unsupervised is almost as good as supervised. If the x_1 .. x_{25} are given the class which was used to learn them, then the results are (μ_1=-2.176, μ_2=1.684). Unsupervised got (μ_1=-2.13, μ_2=1.668).

Finding the max likelihood $\mu_1,\mu_2..\mu_k$

We can compute P(data | $\boldsymbol{\mu}_1,\boldsymbol{\mu}_2..\boldsymbol{\mu}_k$)

How do we find the $\boldsymbol{\mu}_i$'s which give max. likelihood?

- The normal max likelihood trick:

  Set $\frac{\partial}{\partial \mu_i}$ log Prob (….) = 0

  and solve for μ_i's.

  # Here you get non-linear non-analytically- solvable equations

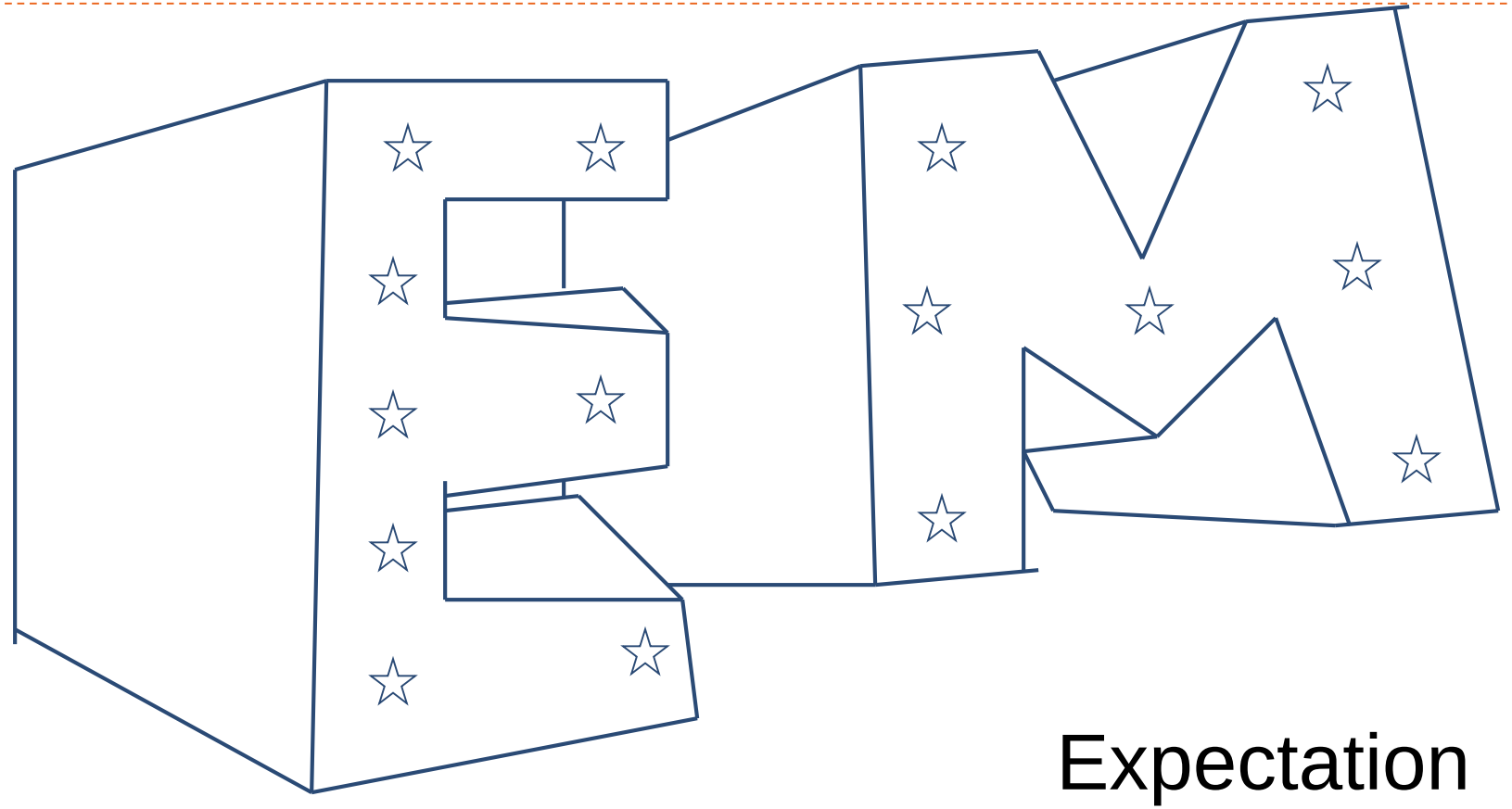
- Use gradient descent

  Slow but doable

- Use a much faster, cuter, and recently very popular method…

Outline

- Gaussian mixture models
- EM algorithm
- Mean-shift algorithm
- KDE
- Self organizing map



Expectation
Maximalization

Silly Example

Let events be "grades in a class"

w_1 = Gets an A P(A) = ½

w_2 = Gets a B P(B) = μ

w_3 = Gets a C P(C) = 2μ

w_4 = Gets a D P(D) = ½-3μ

(Note $0 \leq \mu \leq 1/6$)

Assume we want to estimate μ from data. In a given class there were

a A's
b B's
c C's
d D's

What's the maximum likelihood estimate of μ given a,b,c,d ?

Silly Example

Let events be "grades in a class"

w_1 = Gets an A	P(A) = ½
w_2 = Gets a B	P(B) = μ
w_3 = Gets a C	P(C) = 2μ
w_4 = Gets a D	P(D) = ½-3μ
	(Note $0 \leq \mu \leq 1/6$)

Assume we want to estimate μ from data. In a given class there were

a A's
b B's
c C's
d D's

What's the maximum likelihood estimate of μ given a,b,c,d ?

Trivial Statistics

P(A) = ½ P(B) = μ P(C) = 2μ P(D) = ½-3μ

P(*a*,*b*,*c*,*d* | μ) = K(½)^a(μ)^b(2μ)^c(½-3μ)^d

log P(*a*,*b*,*c*,*d* | μ) = log K + *a*log ½ + *b*log μ + *c*log 2μ + *d*log (½-3μ)

$$\text{FOR MAX LIKE } \mu, \text{SET } \frac{\partial \text{LogP}}{\partial \mu} = 0$$

$$\frac{\partial \text{LogP}}{\partial \mu} = \frac{b}{\mu} + \frac{2c}{2\mu} - \frac{3d}{1/2 - 3\mu} = 0$$

$$\text{Gives max like } \mu = \frac{b+c}{6(b+c+d)}$$

So if class got

A	B	C	D
14	6	9	10

$$\text{Max like } \mu = \frac{1}{10}$$

Boring, but true!

Same Problem with Hidden Information

Someone tells us that

Number of High grades (A's + B's) = h

Number of C's = c

Number of D's = d

What is the max. like estimate of μ now?

REMEMBER

P(A) = ½

P(B) = μ

P(C) = 2μ

P(D) = ½-3μ

Same Problem with Hidden Information

Someone tells us that

Number of High grades (A's + B's) = h

Number of C's = c

Number of D's = d

What is the max. like estimate of μ now?

We can answer this question circularly:

REMEMBER

P(A) = ½

P(B) = μ

P(C) = 2μ

P(D) = ½-3μ

EXPECTATION

If we know the value of μ we could compute the expected value of a and b

Since the ratio a:b should be the same as the ratio ½ : μ

$$a = \frac{1/2}{1/2 + \mu} h \qquad b = \frac{\mu}{1/2 + \mu} h$$

MAXIMIZATION

If we know the expected values of a and b we could compute the maximum likelihood value of μ

$$\mu = \frac{b + c}{6(b + c + d)}$$

E.M. for our Trivial Problem

We begin with a guess for μ

We iterate between EXPECTATION and MAXIMALIZATION to improve our estimates of μ and *a* and *b*.

Define μ(t) the estimate of μ on the t'th iteration

b(t) the estimate of *b* on t'th iteration

REMEMBER

P(A) = ½

P(B) = μ

P(C) = 2μ

P(D) = ½-3μ

$$\mu(0) = \text{initial guess}$$

$$b(t) = \frac{\mu(t)h}{\frac{1}{2} + \mu(t)} = \mathrm{E}[b \mid \mu(t)]$$

E-step

$$\mu(t+1) = \frac{b(t) + c}{6(b(t) + c + d)}$$

$$= \max \text{ like est of } \mu \text{ given } b(t)$$

M-step

Continue iterating until converged.
Good news: Converging to local optimum is assured.
Bad news: I said "local" optimum.

E.M. Convergence

- Convergence proof based on fact that Prob(data | μ) must increase or remain same between each iteration [NOT OBVIOUS]
- But it can never exceed 1 [OBVIOUS]

So it must therefore converge [OBVIOUS]

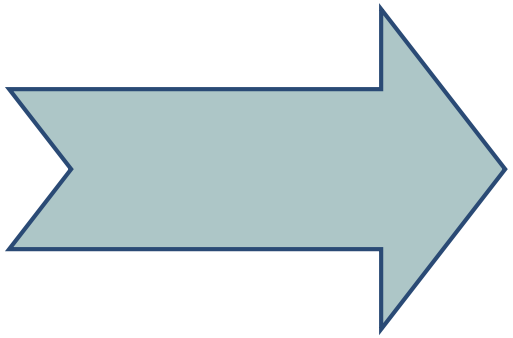
In our example, suppose we had

h = 20

c = 10

d = 10

μ(0) = 0



t	μ(t)	b(t)
0	0	0
1	0.0833	2.857
2	0.0937	3.158
3	0.0947	3.185
4	0.0948	3.187
5	0.0948	3.187
6	0.0948	3.187

Convergence is generally linear: error decreases by a constant factor each time step.

Unsupervised Learning of GMMs

Remember:

We have unlabeled data $\boldsymbol{x}_1\ \boldsymbol{x}_2\ \ldots\ \boldsymbol{x}_R$

We know there are k classes

We know $P(w_1)\ P(w_2)\ P(w_3)\ \ldots\ P(w_k)$

We don't know $\boldsymbol{\mu}_1\ \boldsymbol{\mu}_2\ ..\ \boldsymbol{\mu}_k$

We can write P(data | $\boldsymbol{\mu}_1 \ldots.\ \boldsymbol{\mu}_k$)

$$= p\left(x_1 \ldots x_R \middle| \mu_1 \ldots \mu_k\right)$$

$$= \prod_{i=1}^{R} p\left(x_i \middle| \mu_1 \ldots \mu_k\right)$$

$$= \prod_{i=1}^{R} \sum_{j=1}^{k} p\left(x_i \middle| w_j, \mu_1 \ldots \mu_k\right) P\left(w_j\right)$$

$$= \prod_{i=1}^{R} \sum_{j=1}^{k} K \exp\left(-\frac{1}{2\sigma^2}\left(x_i - \mu_j\right)^2\right) P\left(w_j\right)$$

E.M. for GMMs

For Max likelihood we know $\frac{\partial}{\partial \mu_i} \log \Pr ob\left(data \middle| \mu_1 ... \mu_k\right) = 0$

Some wild' n' crazy algebra turns this into :"For Max likelihood , for each j,

$$\mu_j = \frac{\sum_{i=1}^{R} P\left(w_j \middle| x_i, \mu_1 ... \mu_k\right) x_i}{\sum_{i=1}^{R} P\left(w_j \middle| x_i, \mu_1 ... \mu_k\right)}$$

This is n nonlinear equations in $\boldsymbol{\mu}_j$'s."

If, for each $\mathbf{x}_i$ we knew that for each w_j the prob that $\boldsymbol{\mu}_j$ was in class w_j is $P(w_j|x_i,\mu_1 \ldots \mu_k)$ Then… we would easily compute μ_j.

If we knew each μ_j then we could easily compute $P(w_j|x_i,\mu_1 \ldots \mu_k)$ for each w_j and x_i.

…I feel an EM experience coming on!!

E.M. for GMMs

Iterate. On the *t*'th iteration let our estimates be

$$\lambda_t = \{ \mu_1(t), \mu_2(t) \ldots \mu_c(t) \}$$

E-step

Compute "expected" classes of all datapoints for each class

$$P(w_i | x_k, \lambda_t) = \frac{p(x_k | w_i, \lambda_t) P(w_i | \lambda_t)}{p(x_k | \lambda_t)} = \frac{p(x_k | w_i, \mu_i(t), \sigma^2 \mathbf{I}) p_i(t)}{\sum_{j=1}^{c} p(x_k | w_j, \mu_j(t), \sigma^2 \mathbf{I}) p_j(t)}$$

Just evaluate a Gaussian at x_k

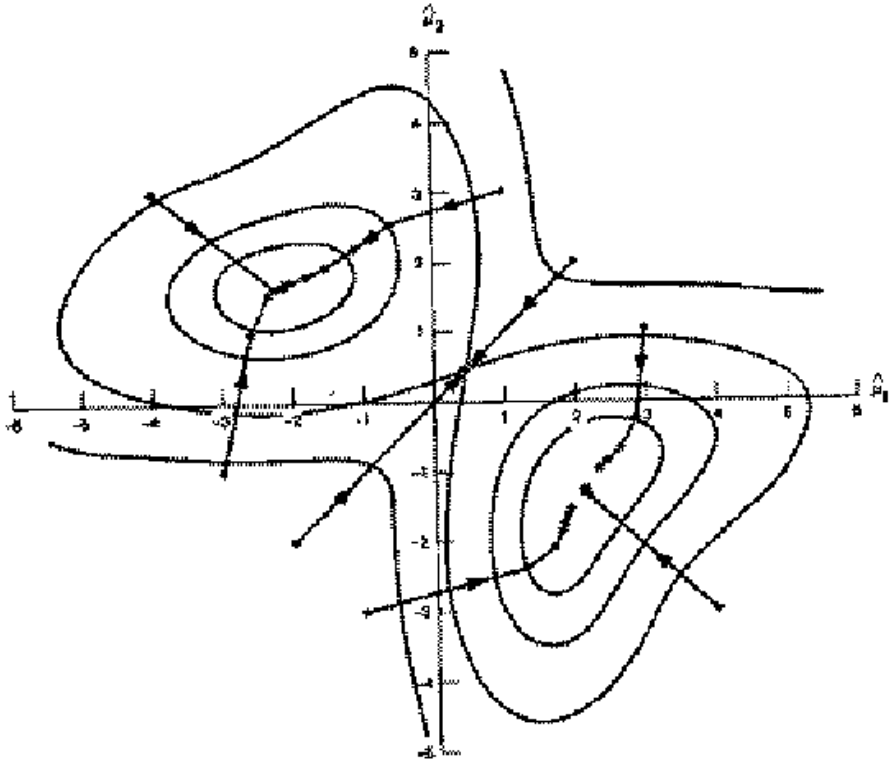
M-step.

Compute Max. like **μ** given our data's class membership distributions

$$\mu_i(t+1) = \frac{\sum_k P(w_i | x_k, \lambda_t) x_k}{\sum_k P(w_i | x_k, \lambda_t)}$$

E.M. Convergence

- This algorithm is REALLY USED. And in high dimensional state spaces, too. E.G. Vector Quantization for Speech Data



E.M. for General GMMs

$p_i(t)$ is shorthand for estimate of $P(\omega_i)$ on t'th iteration

Iterate. On the t'th iteration let our estimates be

$$\lambda_t = \{ \mu_1(t), \mu_2(t) \ldots \mu_c(t), \Sigma_1(t), \Sigma_2(t) \ldots \Sigma_c(t), p_1(t), p_2(t) \ldots p_c(t) \}$$

E-step

Compute "expected" classes of all datapoints for each class

Just evaluate a Gaussian at x_k

$$P(w_i | x_k, \lambda_t) = \frac{p(x_k | w_i, \lambda_t) P(w_i | \lambda_t)}{p(x_k | \lambda_t)} = \frac{p(x_k | w_i, \mu_i(t), \Sigma_i(t)) p_i(t)}{\sum_{j=1}^{c} p(x_k | w_j, \mu_j(t), \Sigma_j(t)) p_j(t)}$$

M-step.

Compute Max. like **μ** given our data's class membership distributions

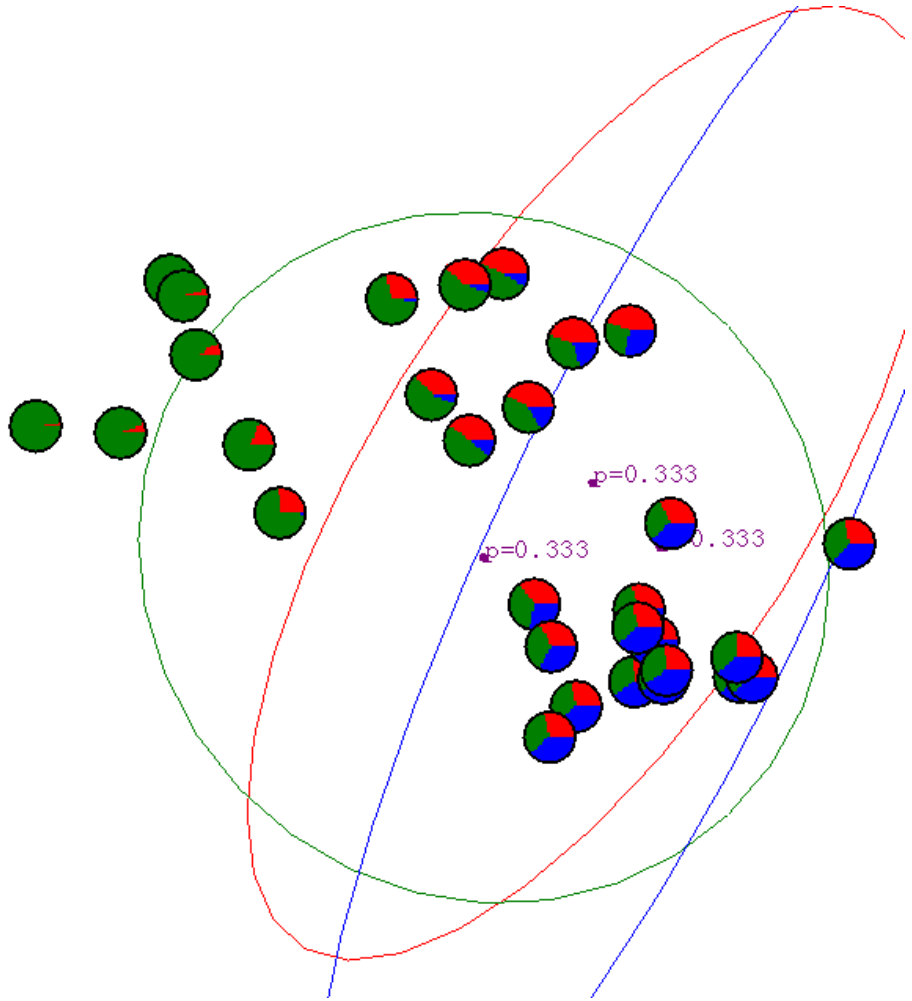
$$\mu_i(t+1) = \frac{\sum_k P(w_i | x_k, \lambda_t) x_k}{\sum_k P(w_i | x_k, \lambda_t)}$$

$$\Sigma_i(t+1) = \frac{\sum_k P(w_i | x_k, \lambda_t) [x_k - \mu_i(t+1)][x_k - \mu_i(t+1)]^T}{\sum_k P(w_i | x_k, \lambda_t)}$$

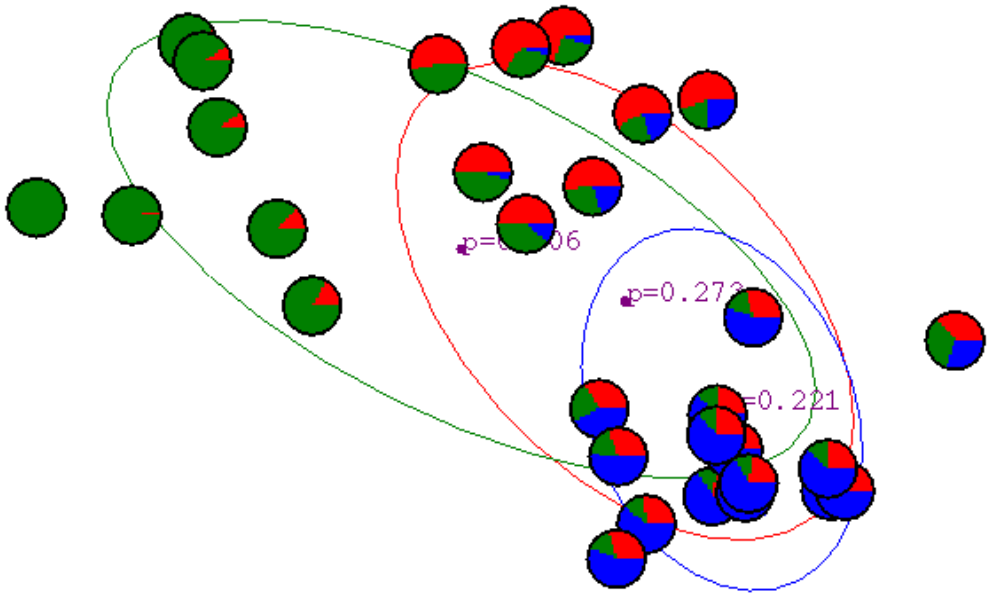
$$p_i(t+1) = \frac{\sum_k P(w_i | x_k, \lambda_t)}{R}$$

R = #records

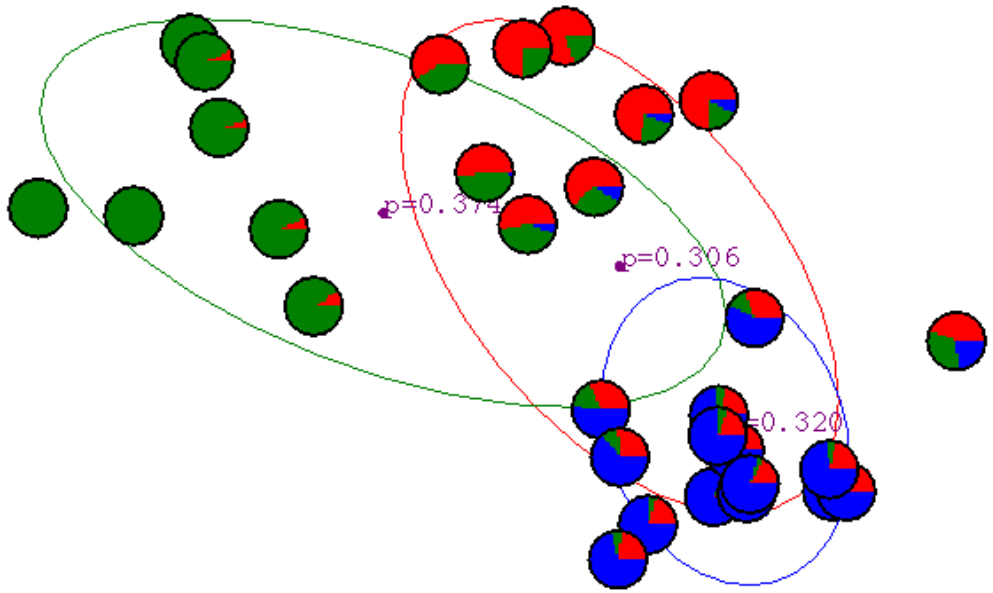
Gaussian Mixture Example: Start



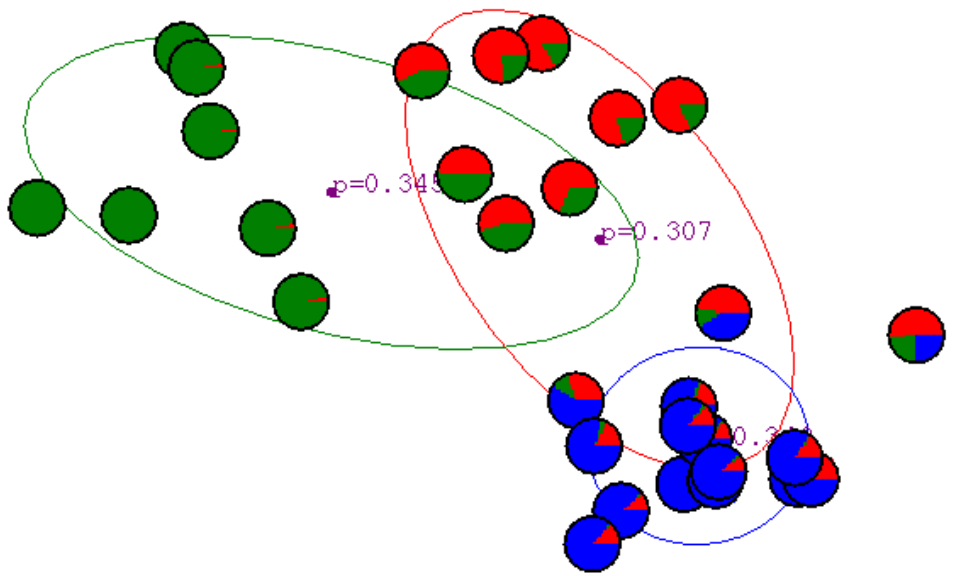
After first iteration



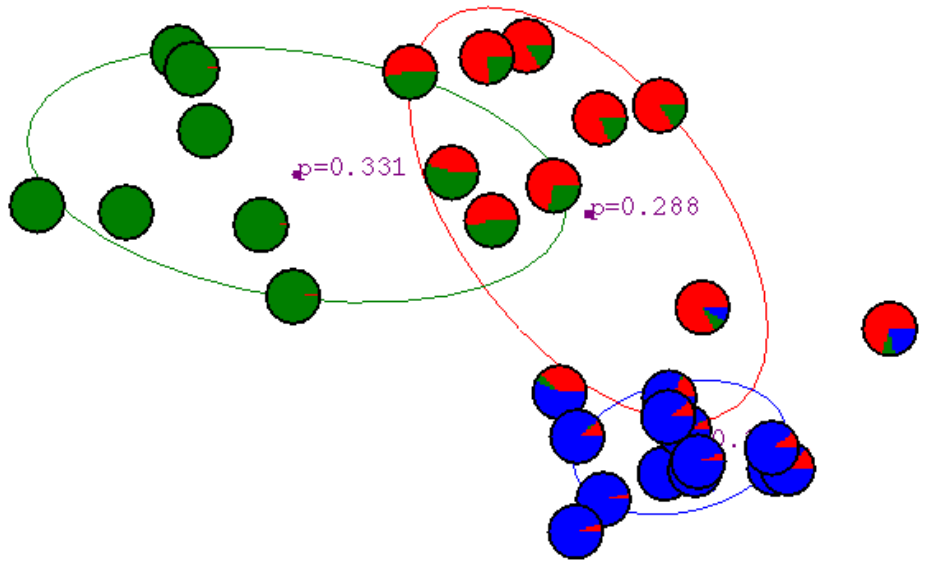
After 2nd iteration



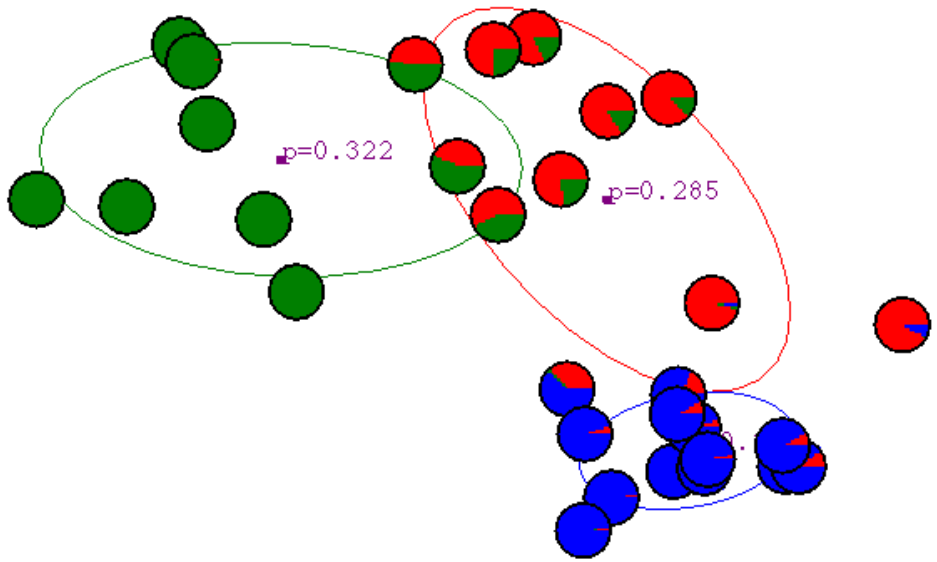
After 3rd iteration



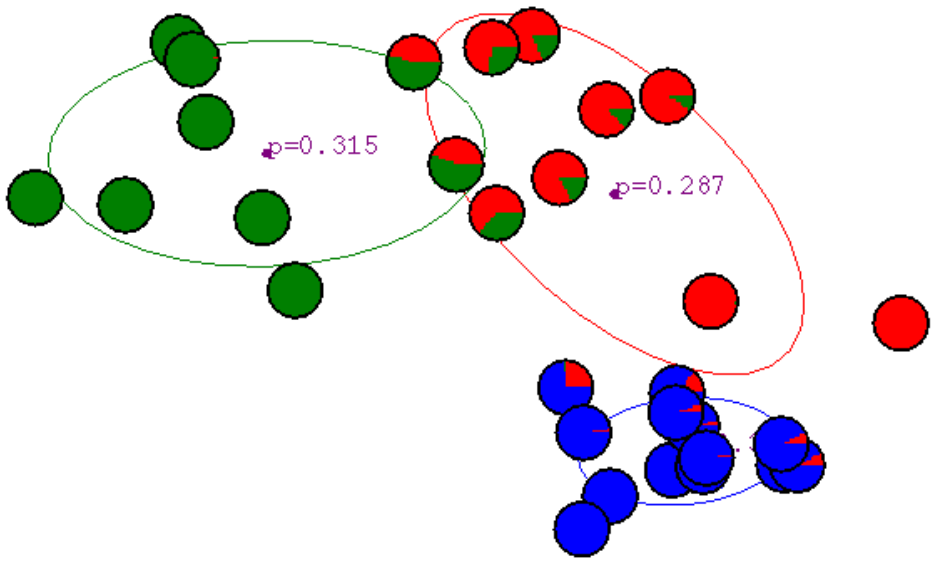
After 4th iteration



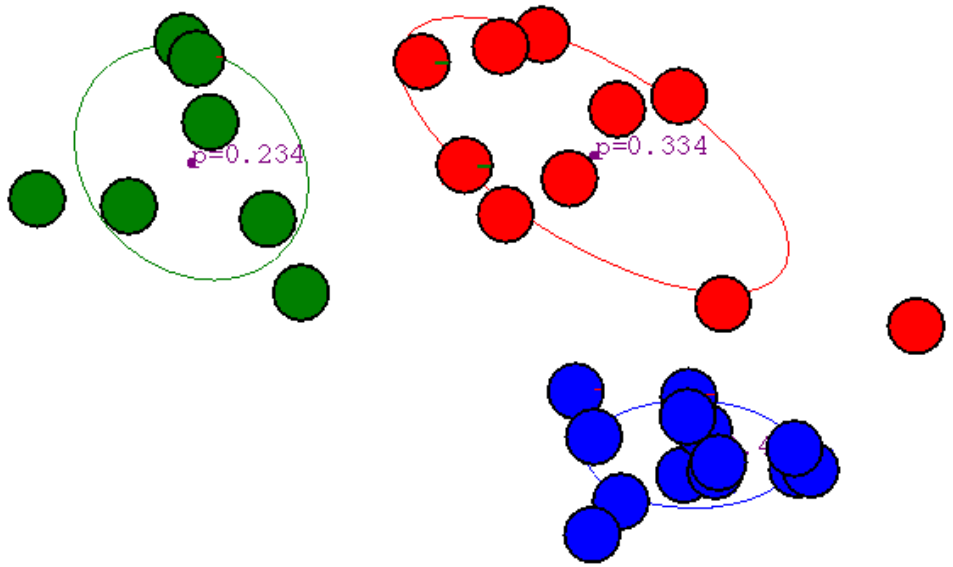
After 5th iteration



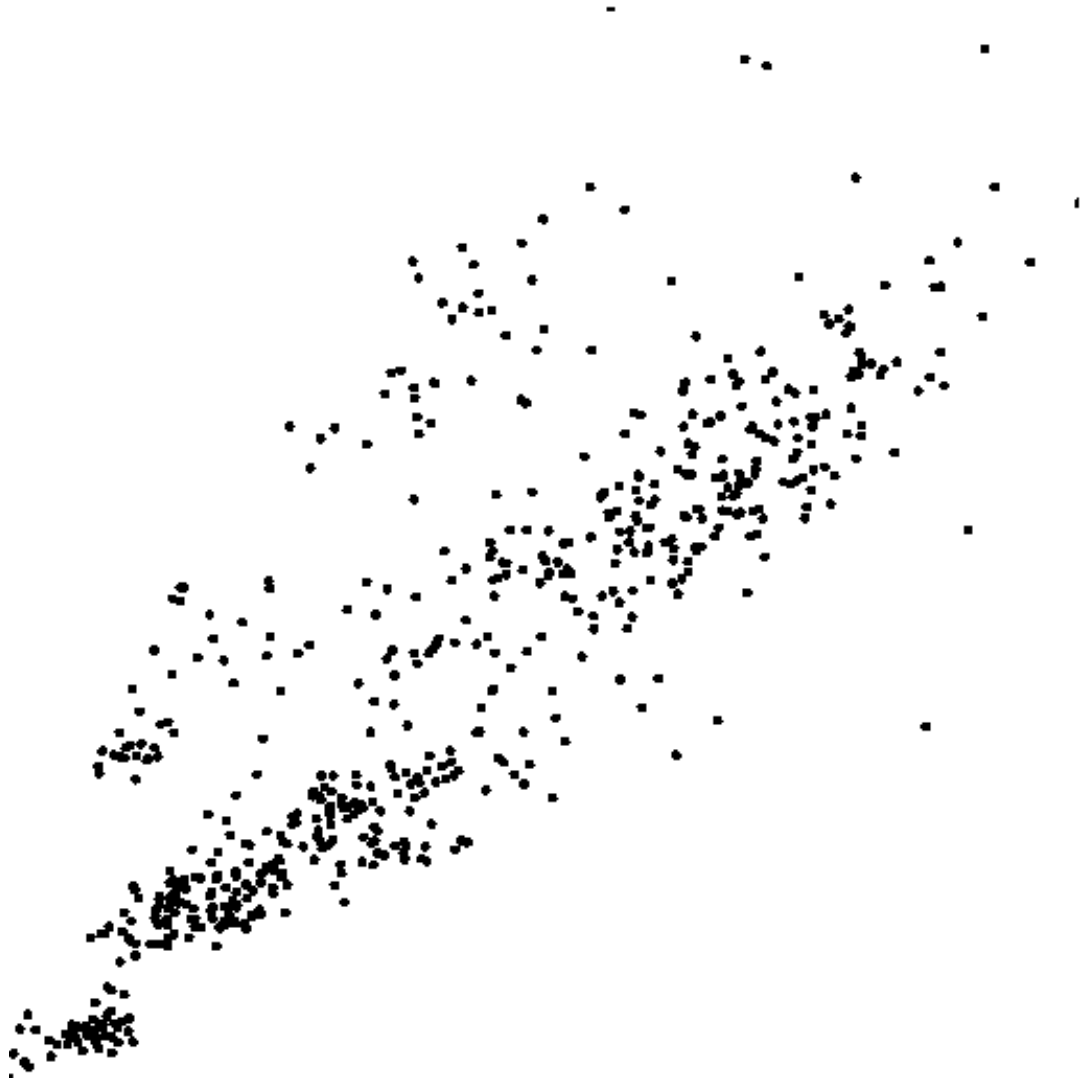
After 6th iteration



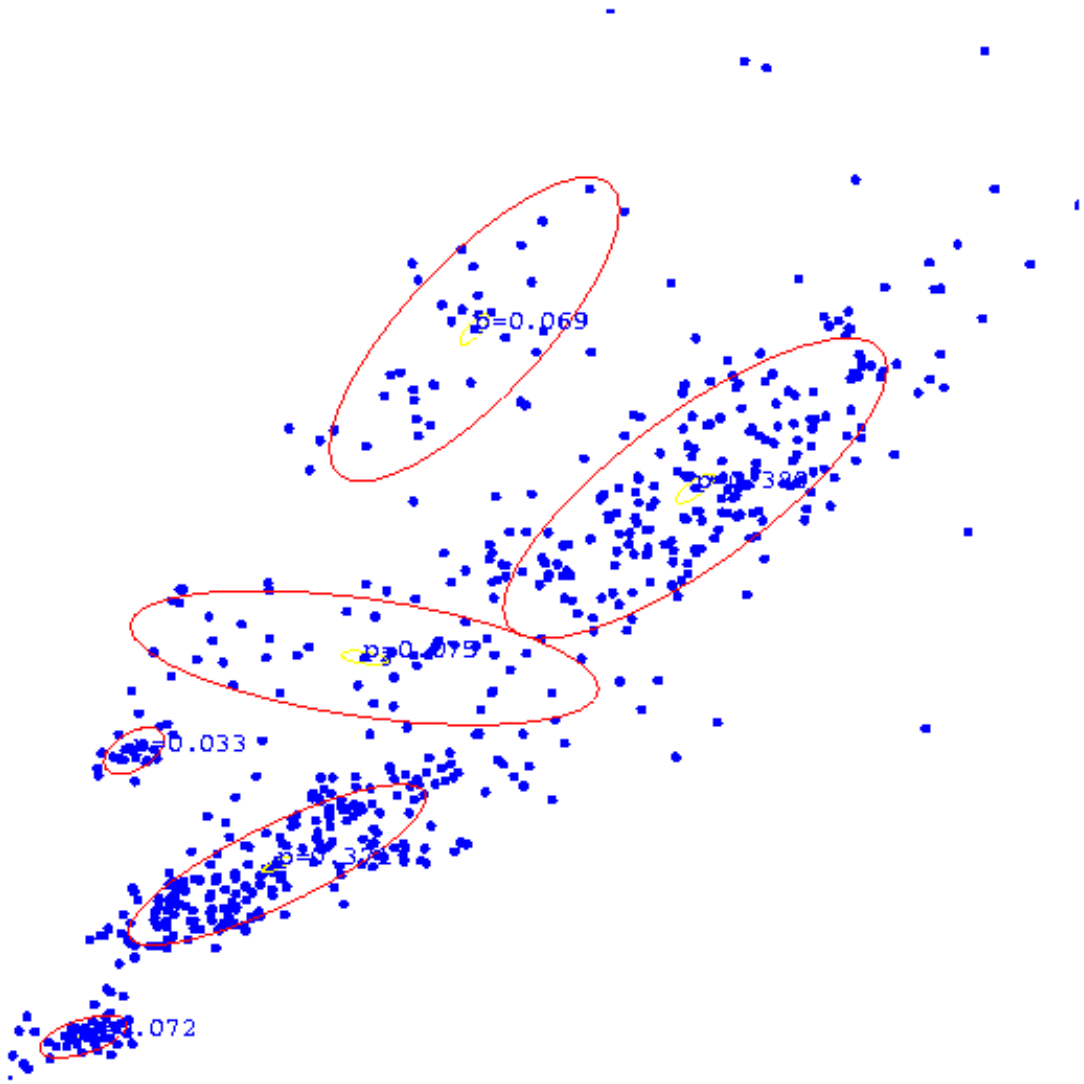
After 20th iteration



Some Bio
Assay data



GMM clustering of the assay data



Outline

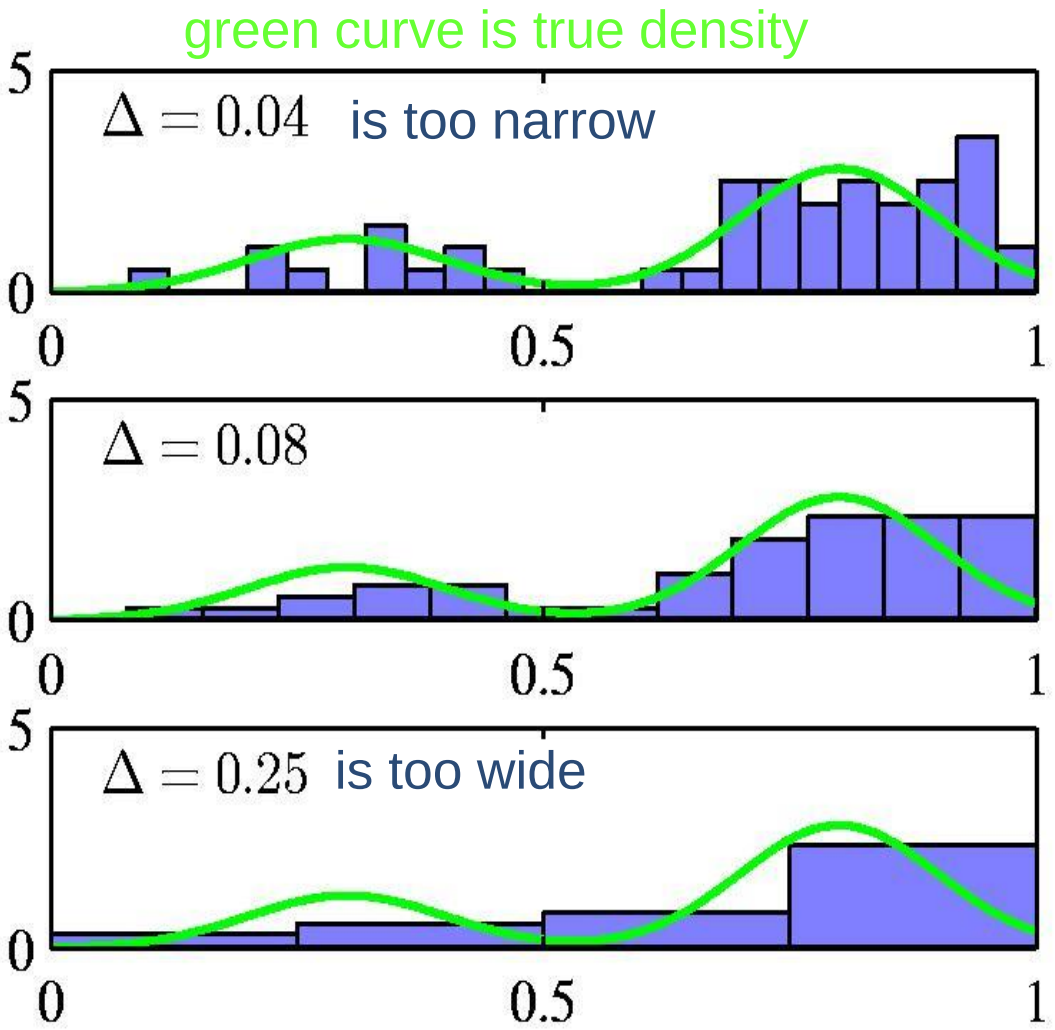
- Gaussian mixture models
- EM algorithm
- Mean-shift algorithm
- KDE
- Self organizing map

Kernel Density Estimation

Chapter 4: UnSupervised Learning

Histograms as density models

- For low dimensional data we can use a histogram as a density model.
  - How wide should the bins be? (width=regulariser)
  - Do we want the same bin-width everywhere?
  - Do we believe the density is zero for empty bins?



$$p_i = \frac{n_i}{N \Delta_i}$$

Histograms as density estimators

- There is no need to fit a model to the data.
  - We just compute some very simple statistics (the number of datapoints in each bin) and store them.
- The number of bins is exponential in the dimensionality of the dataspace. So high-dimensional data is tricky:
  - We must either use big bins or get lots of zero counts (or adapt the local bin-width to the density)
- The density has silly discontinuities at the bin boundaries.
  - We must be able to do better by some kind of smoothing.

Local density estimators

- Estimate the density in a small region to be

$$p(x) = \frac{K}{N\,V}$$

K ← points in region

V ← volume of region

N ↑ total points

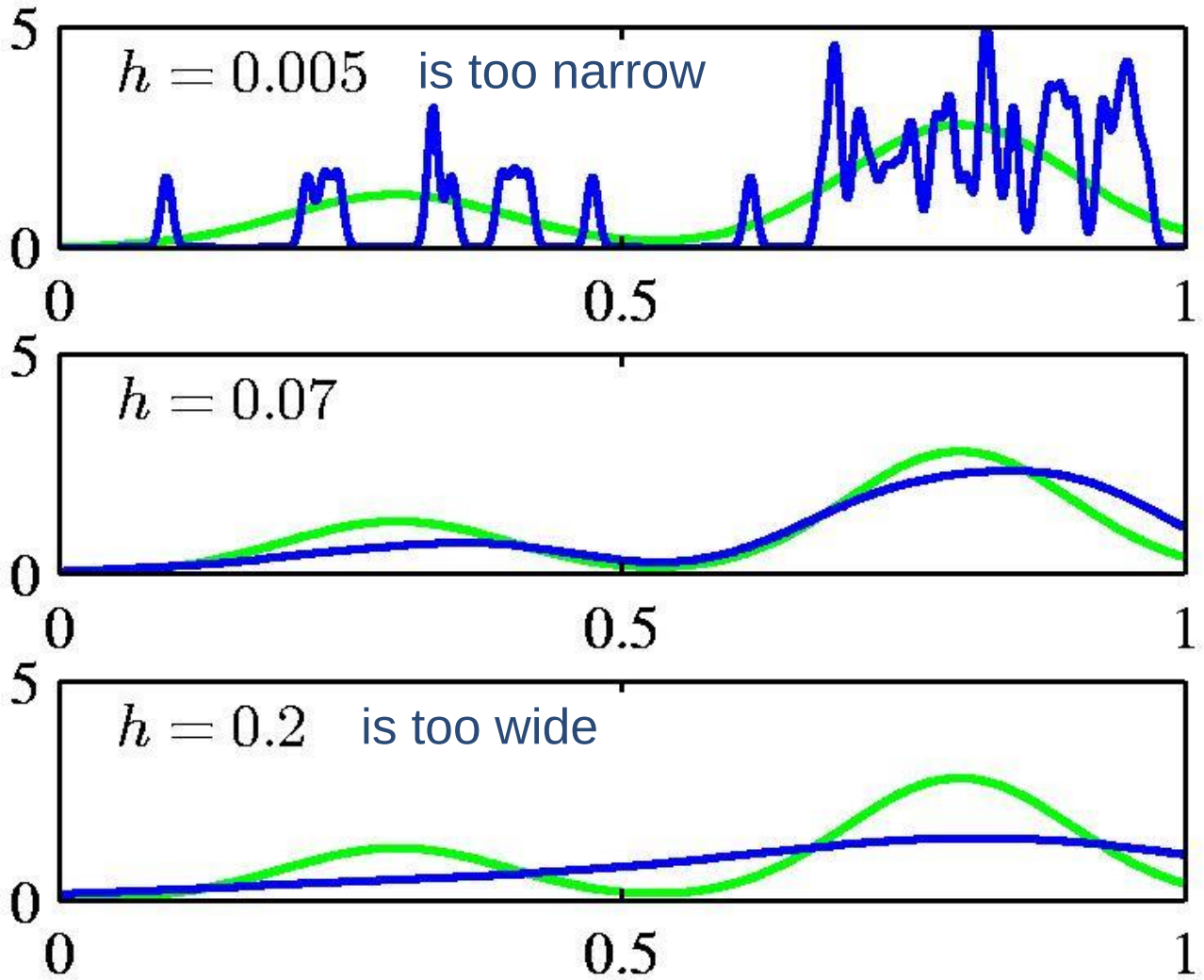
- Problem 1: Variance in estimate if K is small.
- Problem 2: Unmodelled variation across the region if V is big compared with the smoothness of the true density

Kernel density estimators

- Use regions centered on the datapoints
  - Allow the regions to overlap.
  - Let each individual region contribute a total density of 1/N
  - Use regions with soft edges to avoid discontinuities (e.g. isotropic Gaussians)

$$p(\mathbf{x}) = \frac{1}{N}\sum_{n=1}^{N} \frac{1}{(2\pi\sigma^2)^{D/2}} \exp\left(\frac{\| \mathbf{x}-\mathbf{x}_n \|^2}{2\sigma^2}\right)$$

The density modeled by a kernel density estimator



Nearest neighbor methods for density estimation

$$p(x) = \frac{K}{N\,V}$$

K ← points in region

V ← volume of region

N ↑ total points

- Vary the size of a hyper-sphere around each test point so that exactly K training datapoints fall inside the hyper-sphere.
  - Does this give a fair estimate of the density?
- Nearest neighbors is usually used for classification or regression:
  - For regression, average the predictions of the K nearest neighbors.
  - For classification, pick the class with the most votes.
    - How should we break ties?

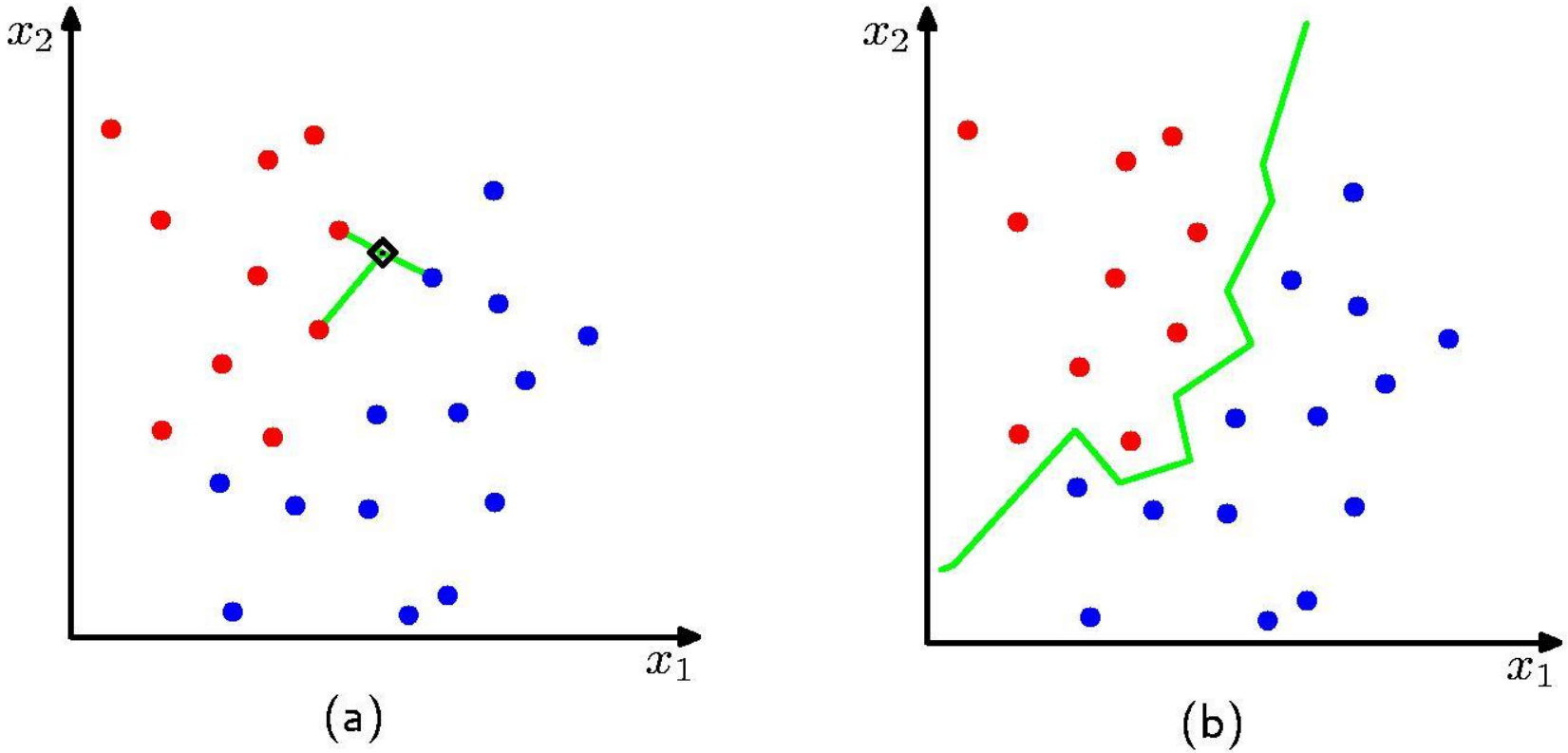
Nearest neighbor methods for classification and regression

- Nearest neighbors is usually used for classification or regression:
- For regression, average the predictions of the K nearest neighbors.
  - How should we pick K?
- For classification, pick the class with the most votes.
  - How should we break ties?
  - Let the k'th nearest neighbor contribute a count that falls off with k. For example,

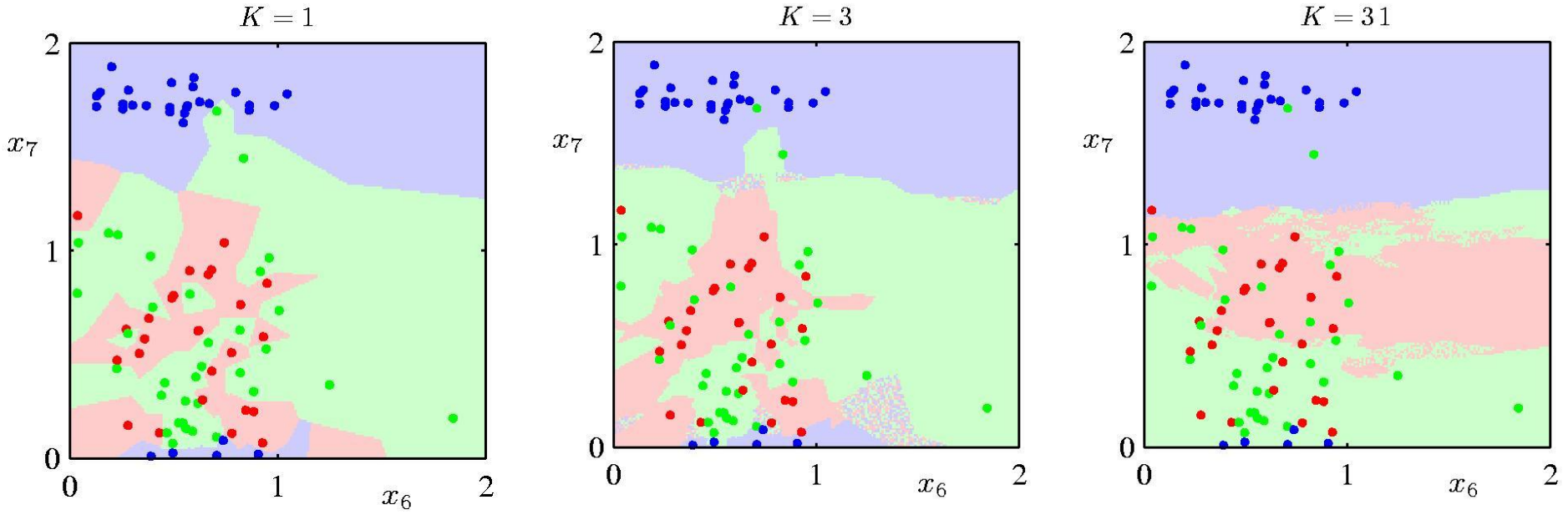
$$1+\frac{1}{2^k}$$

The decision boundary implemented by 3NN

The boundary is always the perpendicular bisector of the line between two points (Vornoi tesselation)



Regions defined by using various numbers of neighbors



Outline

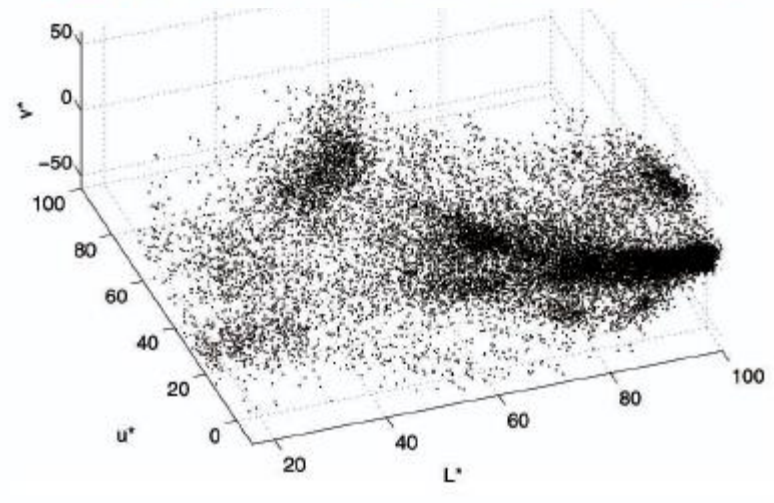
- Gaussian mixture models
- EM algorithm
- Mean-shift algorithm
- KDE
- Self organizing map

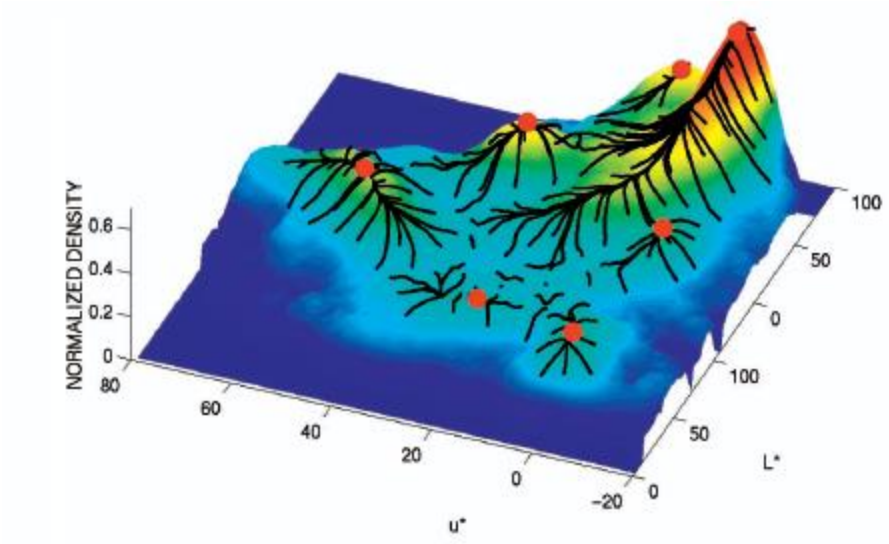
Mean Shift Algorithm

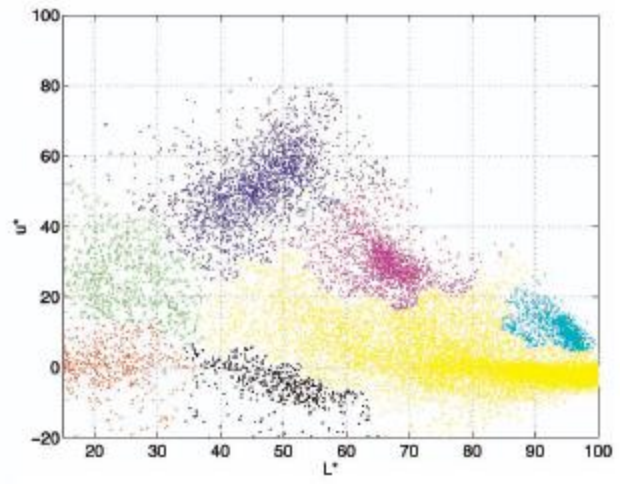
Chapter 4: UnSupervised Learning

Mean shift algorithm

- Try to find *modes* of this non-parametric density







Kernel density estimation

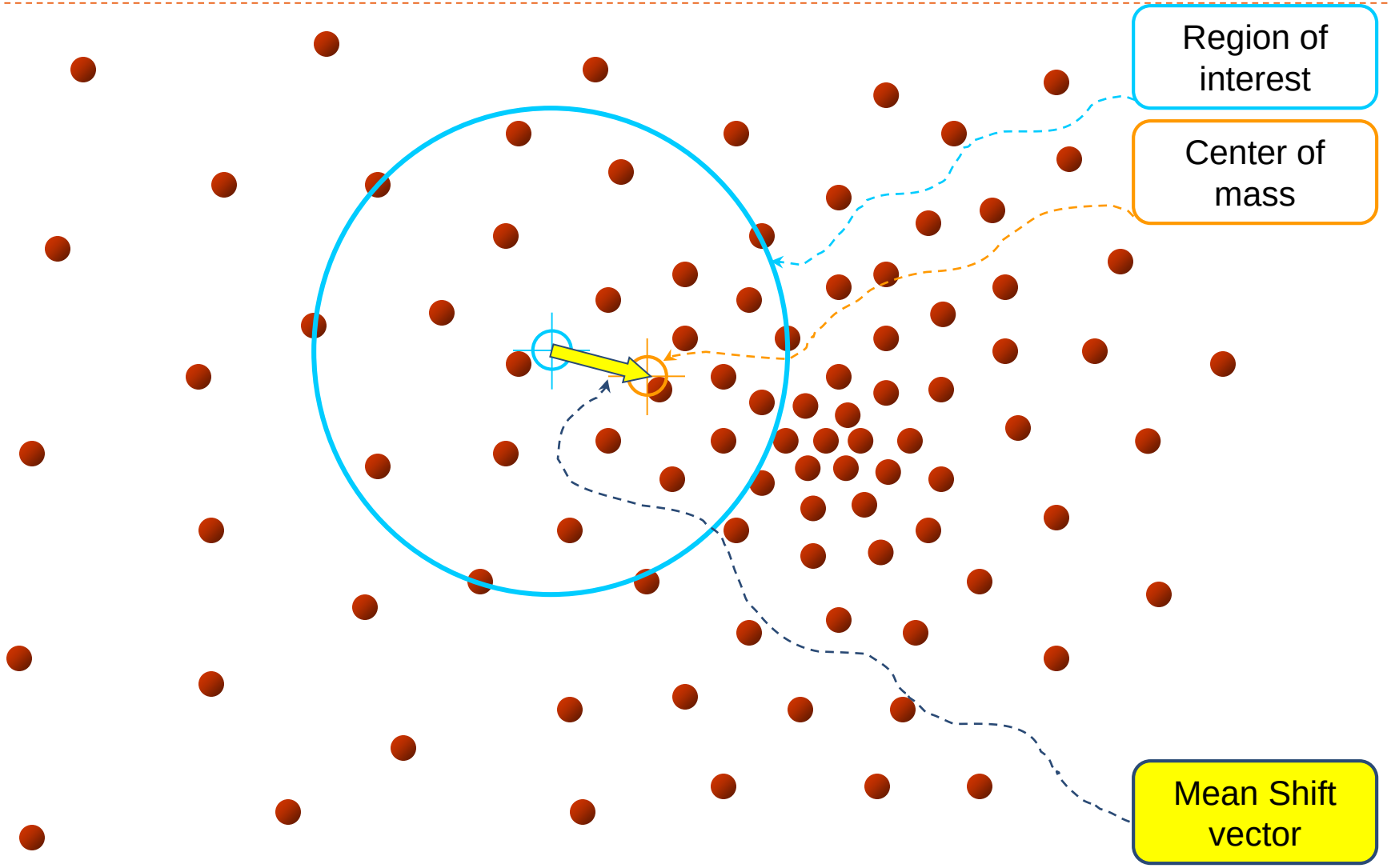
Kernel density estimation function

$$\widehat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

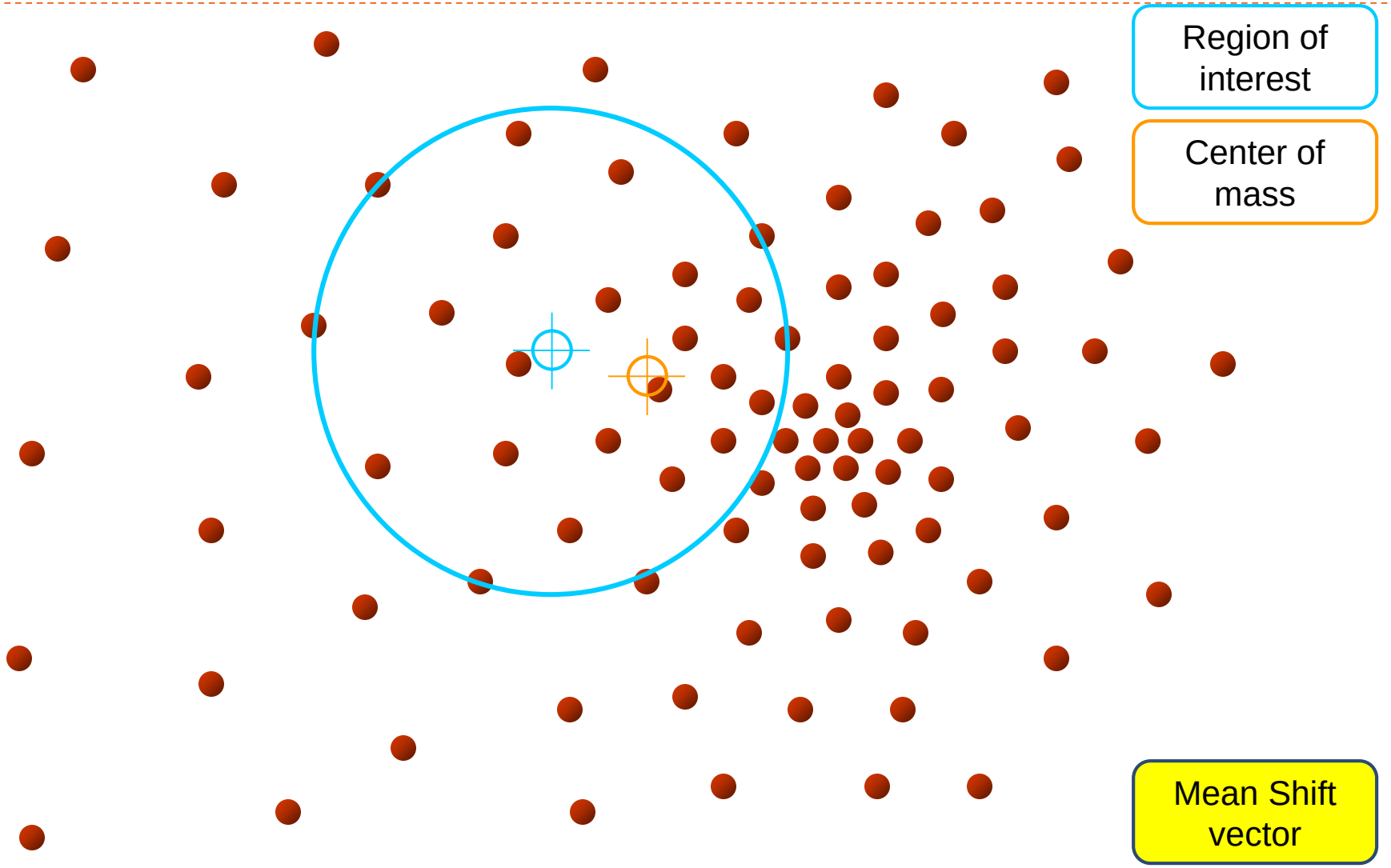
Gaussian kernel

$$K\left(\frac{x - x_i}{h}\right) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-x_i)^2}{2h^2}}.$$

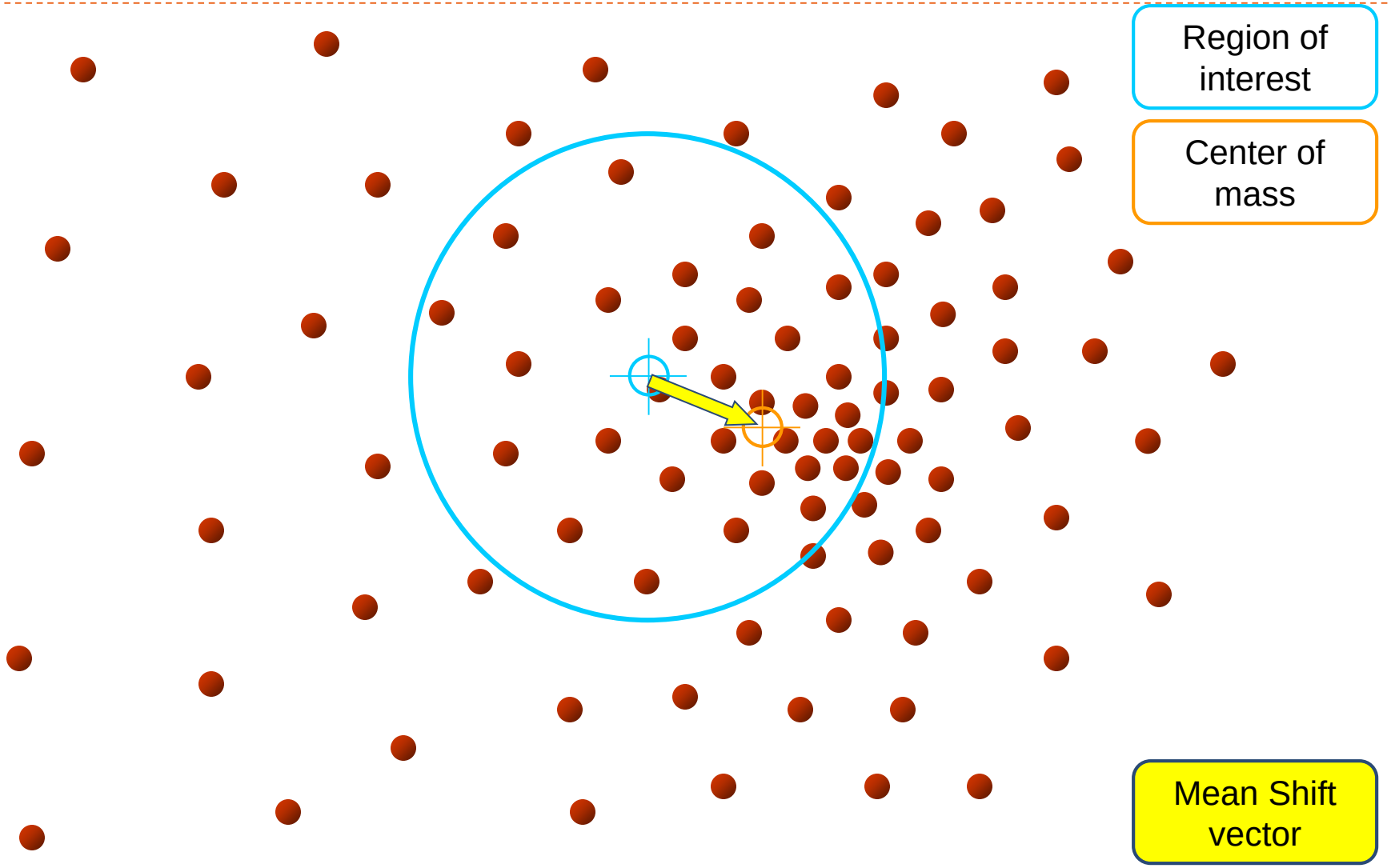
Mean shift



Mean shift

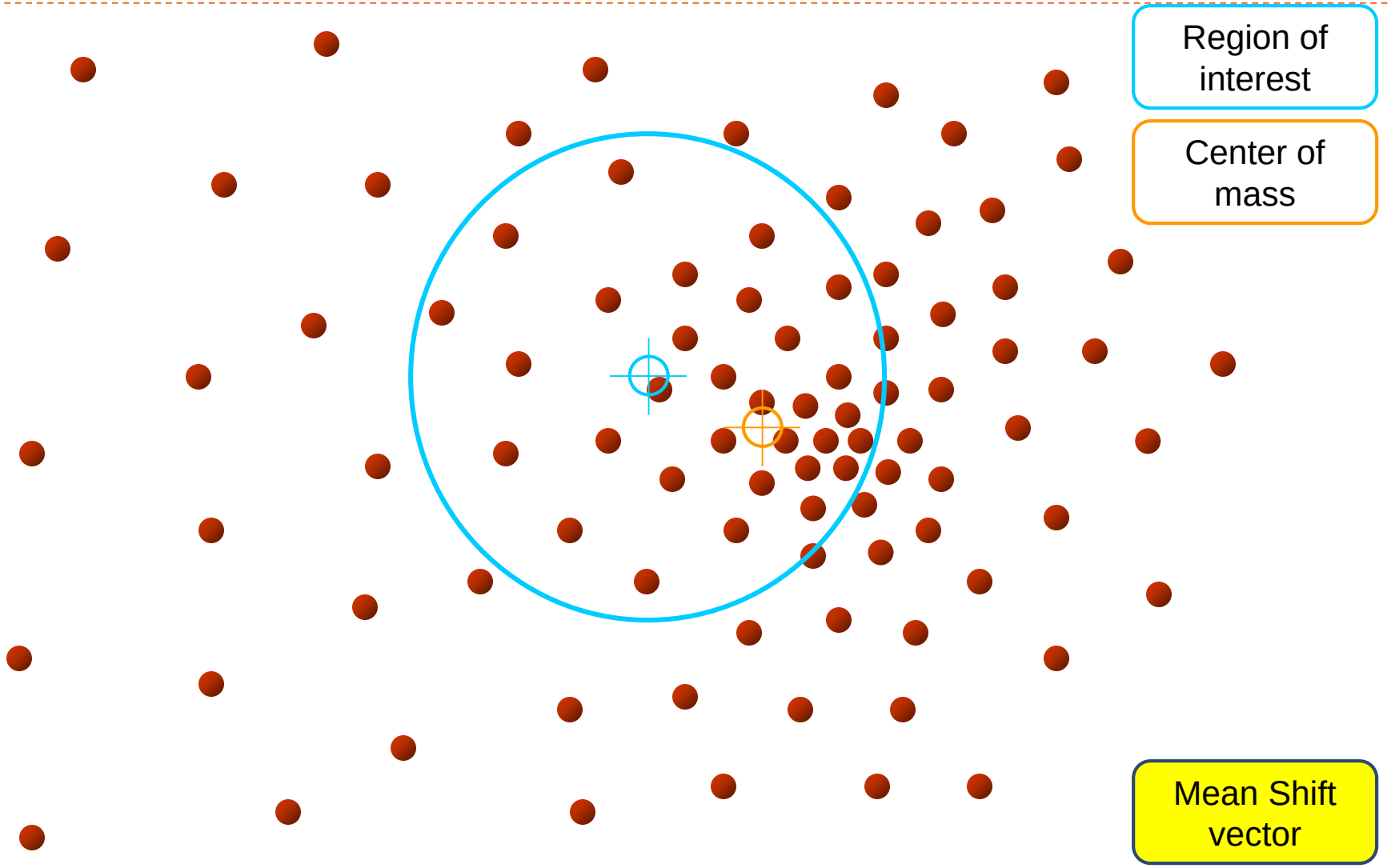


Mean shift

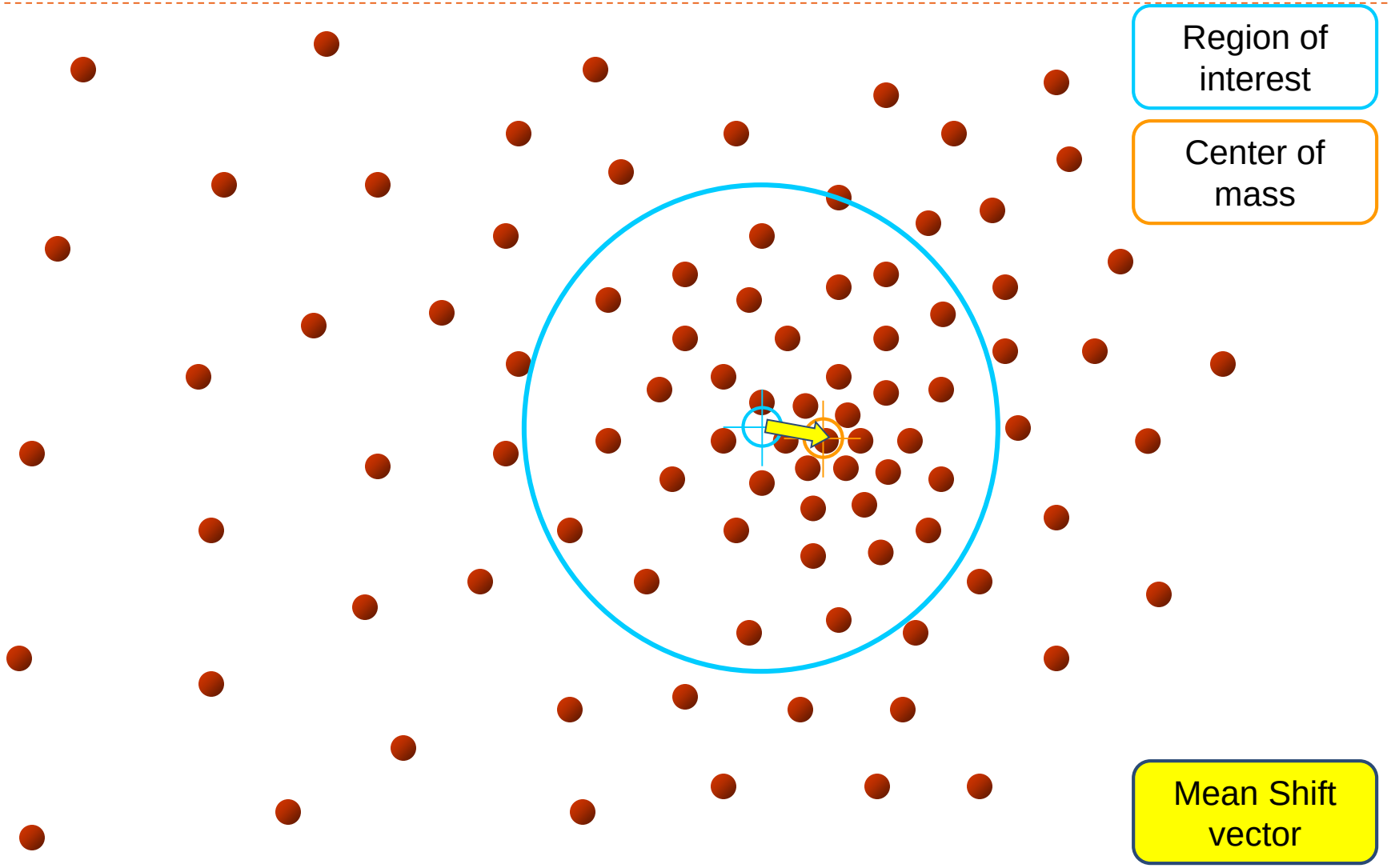


Slide by Y. Ukrainitz & B. Sarel

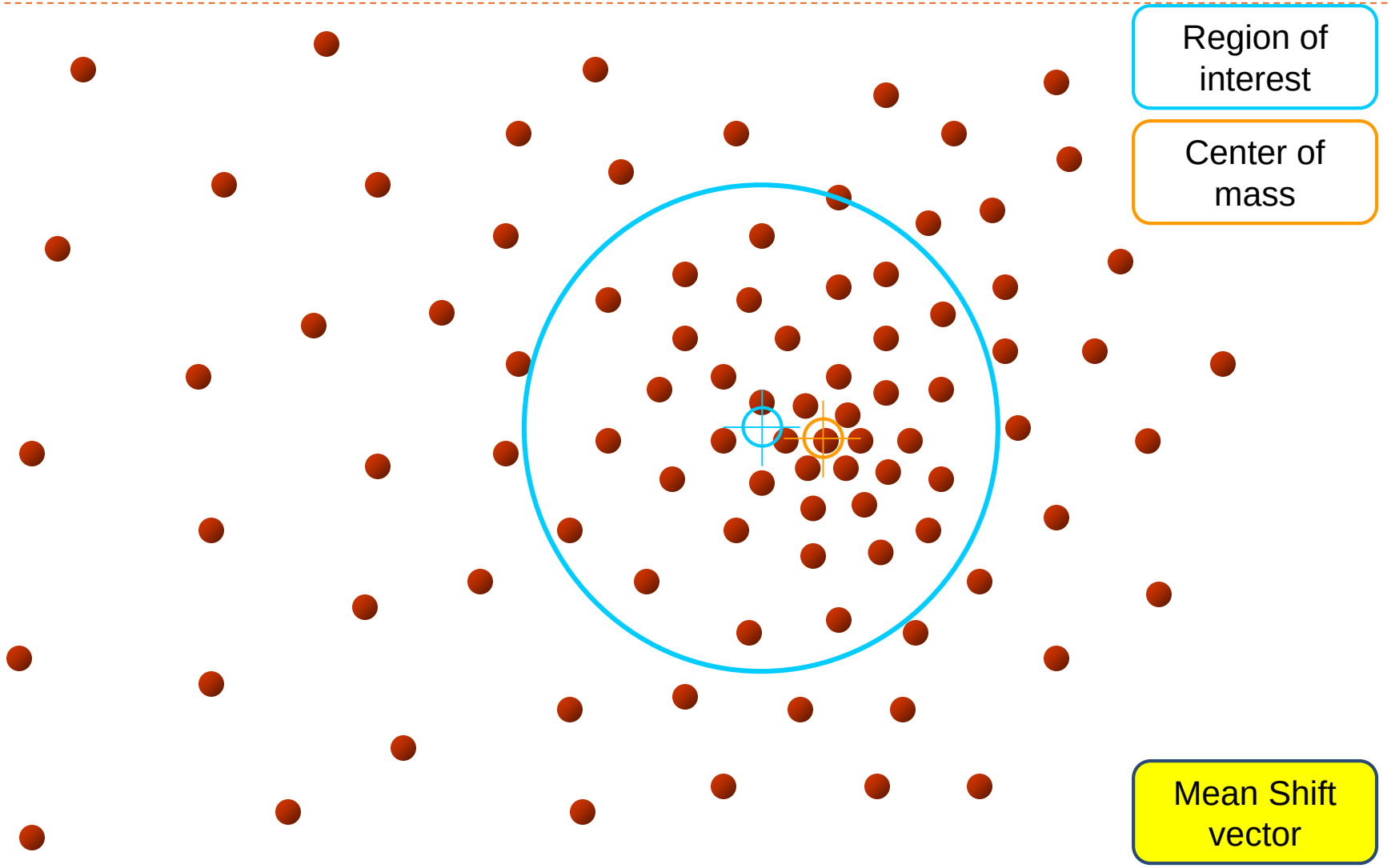
Mean shift



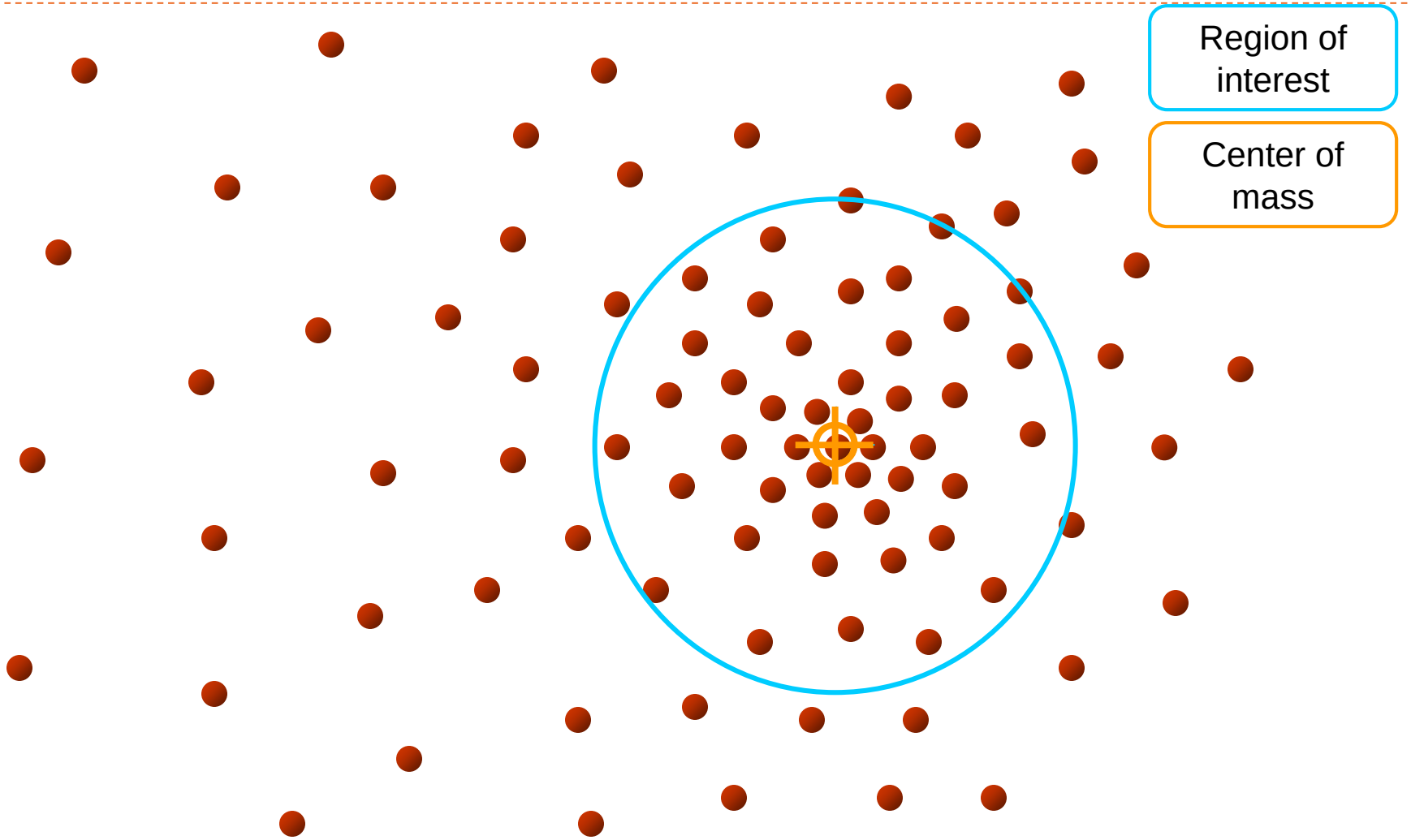
Mean shift



Mean shift



Mean shift



Computing the Mean Shift

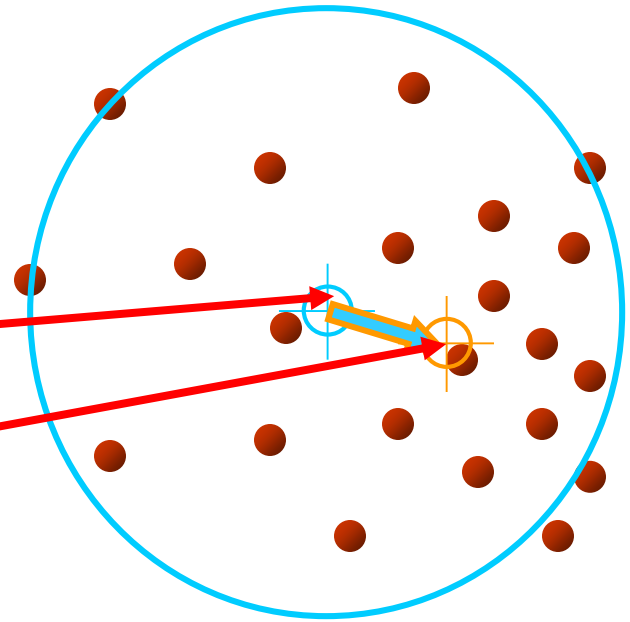
Simple Mean Shift procedure:

- Compute mean shift vector

- Translate the Kernel window by **m(x)**

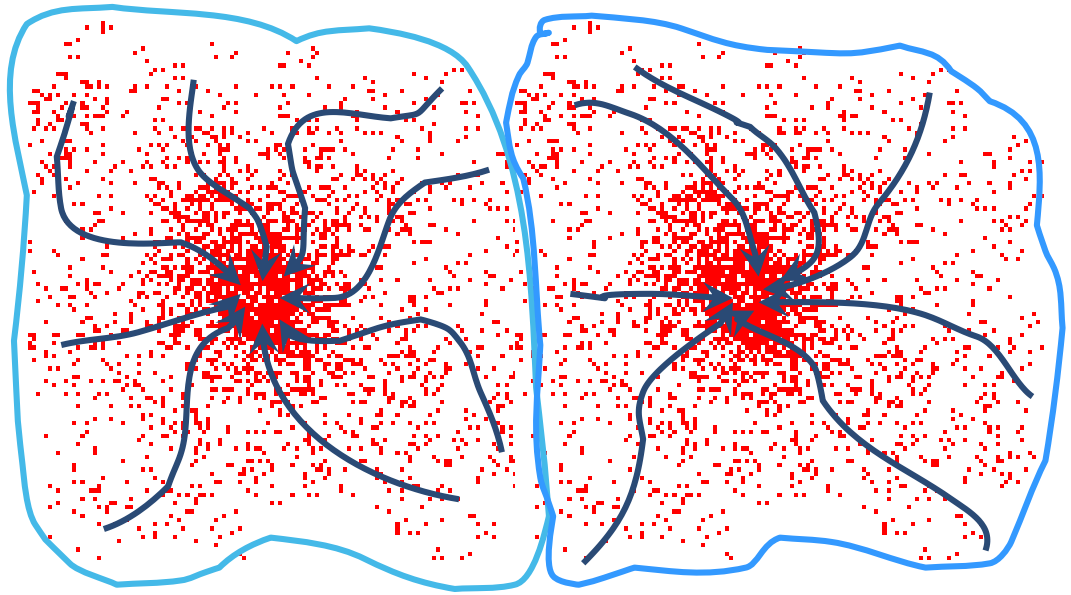
$$g(\mathbf{x}) = -k'(\mathbf{x})$$

$$\mathbf{m}(\mathbf{x}) = \left[\frac{\sum_{i=1}^{n} \mathbf{x}_i g\left(\frac{\|\mathbf{x} - \mathbf{x}_i\|^2}{h} \right)}{\sum_{i=1}^{n} g\left(\frac{\|\mathbf{x} - \mathbf{x}_i\|^2}{h} \right)} \right] - \mathbf{x}$$

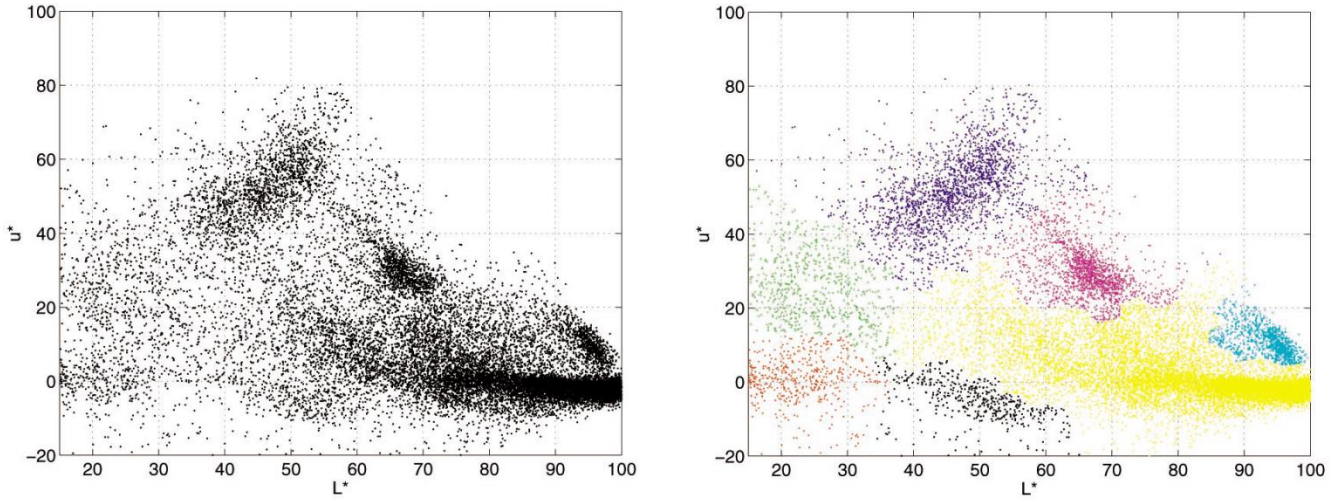


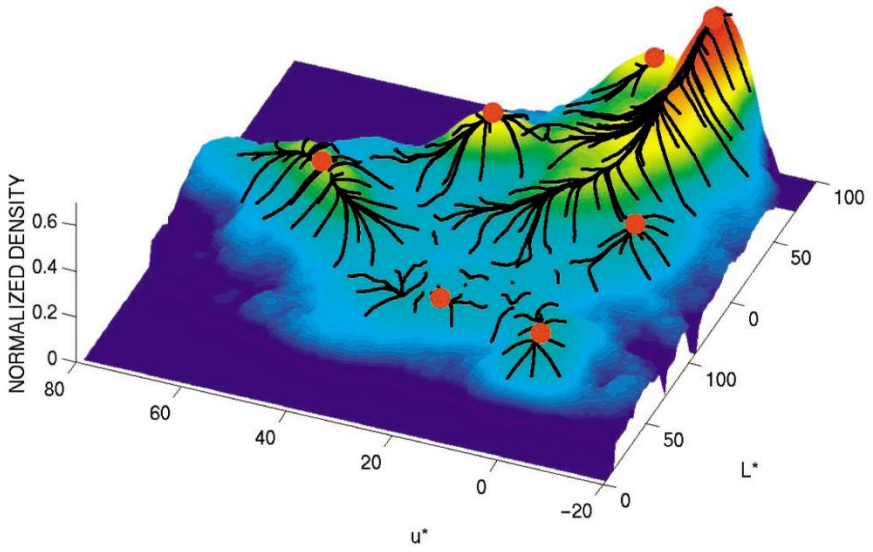
Attraction basin

- Attraction basin: the region for which all trajectories lead to the same mode
- Cluster: all data points in the attraction basin of a mode



Attraction basin



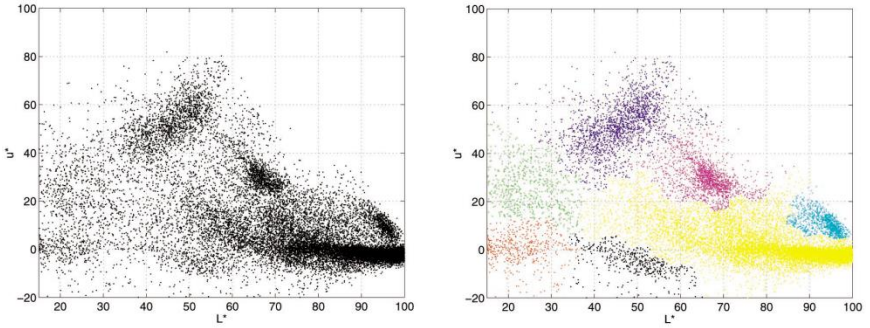


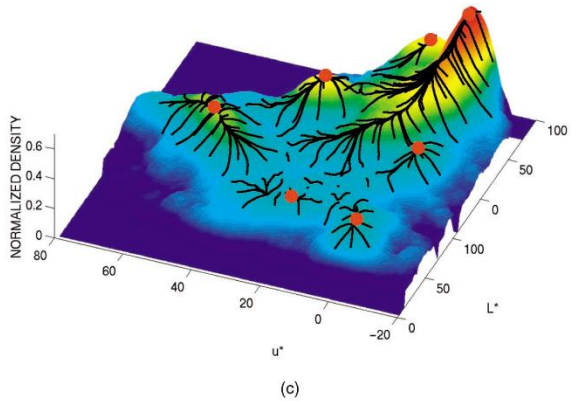
Mean shift clustering

- The mean shift algorithm seeks *modes* of the given set of points
  1. Choose kernel and bandwidth
  2. For each point:
     a) Center a window on that point
     b) Compute the mean of the data in the search window
     c) Center the search window at the new mean location
     d) Repeat (b,c) until convergence
  3. Assign points that lead to nearby modes to the same cluster

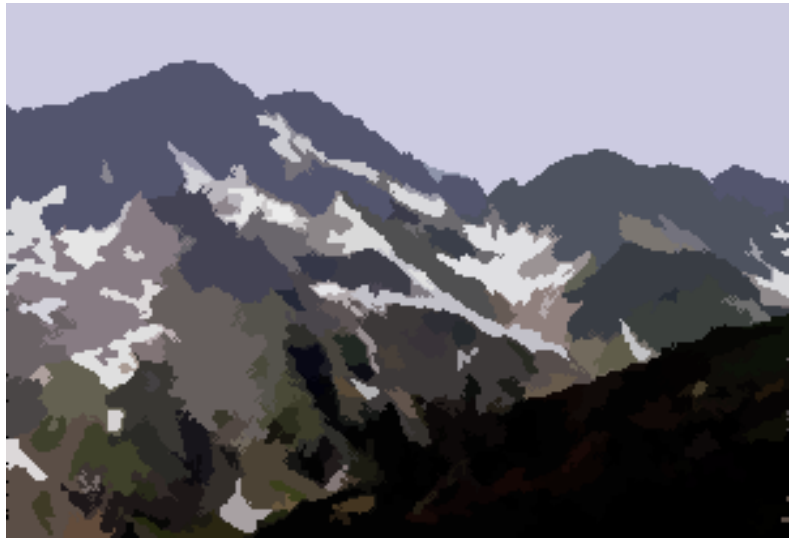
Segmentation by Mean Shift

- Compute features for each pixel (color, gradients, texture, etc)
- Set kernel size for features K_f and position K_s
- Initialize windows at individual pixel locations
- Perform mean shift for each window until convergence
- Merge windows that are within width of K_f and K_s

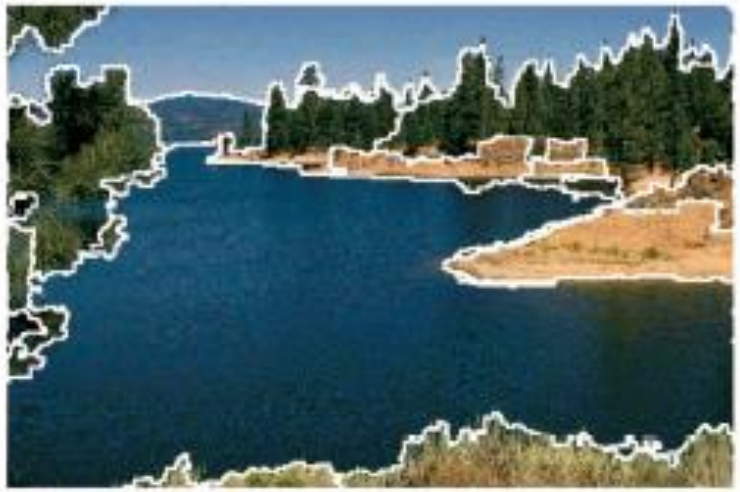




Mean shift segmentation



http://www.caip.rutgers.edu/~comanici/MSPAMI/msPamiResults.html



Mean-shift: issues

- Speedups
  - Binned estimation
  - Fast search of neighbors
  - Update each window in each iteration (faster convergence)

- Other tricks
  - Use kNN to determine window sizes adaptively

- Lots of theoretical support

  D. Comaniciu and P. Meer, Mean Shift: A Robust Approach toward Feature Space Analysis, PAMI 2002.

Mean shift pros and cons

- Pros
  - Good general-practice segmentation
  - Flexible in number and shape of regions
  - Robust to outliers
- Cons
  - Have to choose kernel size in advance
  - Not suitable for high-dimensional features
- When to use it
  - Oversegmentatoin
  - Multiple segmentations
  - Tracking, clustering, filtering applications

Outline

- Gaussian mixture models
- EM algorithm
- Mean-shift algorithm
- KDE
- Self organizing map

Self organizing maps

- 1. What is a Self Organizing Map?
- 2. Topographic Maps
- 3. Setting up a Self Organizing Map
- 4. Kohonen Networks
- 5. Components of Self Organization
- 6. Overview of the SOM Algorithm

What is a self organizing map?

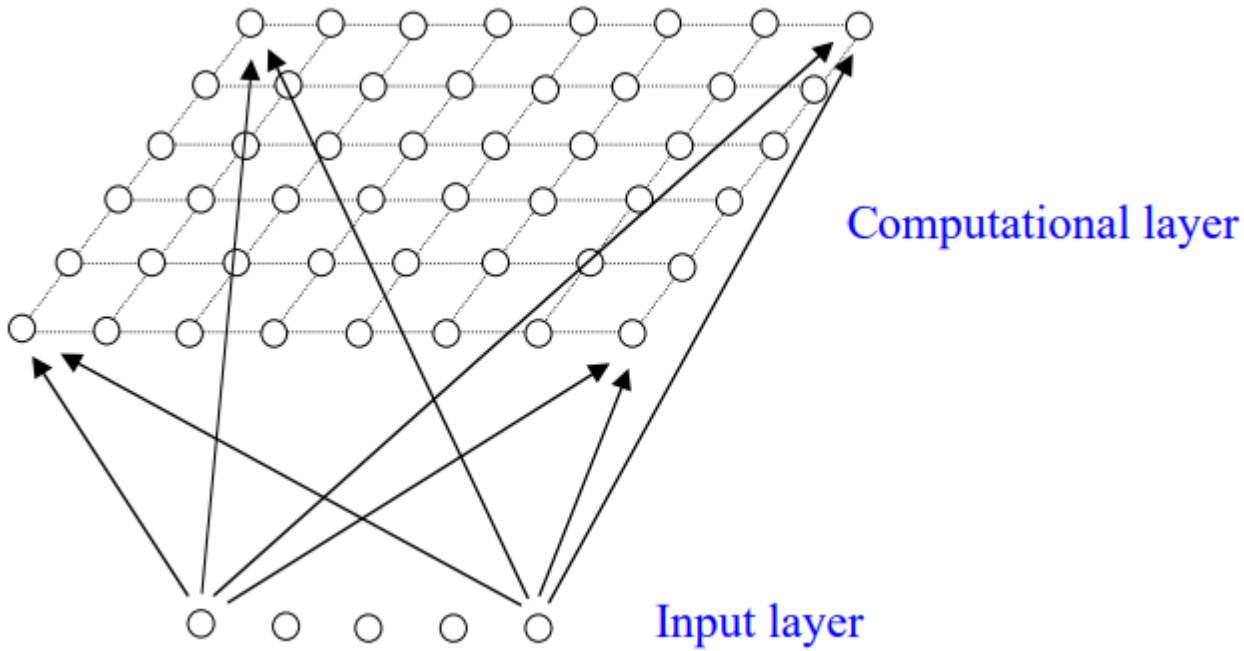
- One particularly interesting class of unsupervised system is based on competitive learning, in which the output neurons compete amongst themselves to be activated, with the result that only one is activated at any one time.
- This activated neuron is called a winner-takes all neuron or simply the winning neuron. Such competition can be induced/implemented by having lateral inhibition connections (negative feedback paths) between the neurons.
- The result is that the neurons are forced to organise themselves. For obvious reasons, such a network is called a Self Organizing Map (SOM)

Set up SOM

- The principal goal of an SOM is to transform an incoming signal pattern of arbitrary dimension into a one or two dimensional discrete map, and to perform this transformation adaptively in a topologically ordered fashion.
- We therefore set up our SOM by placing neurons at the nodes of a one or two dimensional lattice. Higher dimensional maps are also possible, but not so common.
- The neurons become selectively tuned to various input patterns (stimuli) or classes of
- input patterns during the course of the competitive learning.
- The locations of the neurons so tuned (i.e. the winning neurons) become ordered and a meaningful coordinate system for the input features is created on the lattice. The SOM thus forms the required topographic map of the input patterns.

The Architecture a Self Organizing Map

- We shall concentrate on the SOM system known as a Kohonen Network. This has a feed-forward structure with a single computational layer of neurons arranged in rows and columns. Each neuron is fully connected to all the source units in the input layer:



- A one dimensional map will just have a single row or column in the computational layer.

Components of Self Organization

- The self-organization process involves four major components:

- Initialization: All the connection weights are initialized with small random values.

- Competition: For each input pattern, the neurons compute their respective values of a discriminant function which provides the basis for competition. The particular neuron with the smallest value of the discriminant function is declared the winner.

- Cooperation: The winning neuron determines the spatial location of a topological neighbourhood of excited neurons, thereby providing the basis for cooperation among neighbouring neurons.

- Adaptation: The excited neurons decrease their individual values of the discriminant function in relation to the input pattern through suitable adjustment of the associated connection weights, such that the response of the winning neuron to the subsequent application of a similar input pattern is enhanced

The Competitive Process

- If the input space is D dimensional (i.e. there are D input units) we can write the input patterns as x = $\{x_i : i = 1, \ldots, D\}$ and the connection weights between the input units i and the neurons j in the computation layer can be written $w_j = \{w_{ji} : j = 1, \ldots, N; i = 1, \ldots, D\}$ where N is the total number of neurons.

- We can then define our discriminant function to be the squared Euclidean distance between the input vector x and the weight vector wj for each neuron j

$$d_j(\mathbf{x}) = \sum_{i=1}^{D} (x_i - w_{ji})^2$$

- In other words, the neuron whose weight vector comes closest to the input vector (i.e. is most similar to it) is declared the winner.

- In this way the continuous input space can be mapped to the discrete output space of neurons by a simple process of competition between the neurons

The Cooperative Process

- In neurobiological studies we find that there is lateral interaction within a set of excited neurons. When one neuron fires, its closest neighbours tend to get excited more than those further away. There is a topological neighbourhood that decays with distance.

- We want to define a similar topological neighbourhood for the neurons in our SOM. If Sij is the lateral distance between neurons i and j on the grid of neurons, we take

$$T_{j,I(\mathbf{x})} = \exp(-S_{j,I(\mathbf{x})}^2 / 2\sigma^2)$$

- as our topological neighbourhood, where I(x) is the index of the winning neuron. This has several important properties: it is maximal at the winning neuron, it is symmetrical about that neuron, it decreases monotonically to zero as the distance goes to infinity, and it is translation invariant (i.e. independent of the location of the winning neuron)

- A special feature of the SOM is that the size σ of the neighbourhood needs to decrease with time. A popular time dependence is an exponential decay: $\sigma(t) = \sigma_0 \exp(-t / \tau_\sigma)$

92

The Adaptive Process

- Clearly our SOM must involve some kind of adaptive, or learning, process by which the outputs become self-organised and the feature map between inputs and outputs is formed.

- The point of the topographic neighbourhood is that not only the winning neuron gets its weights updated, but its neighbours will have their weights updated as well, although by not as much as the winner itself. In practice, the appropriate weight update equation is

$$\Delta w_{ji} = \eta(t) \, . \, T_{j,I(\mathbf{x})}(t) \, . \, (x_i - w_{ji})$$

- in which we have a time (epoch) t dependent learning rate η: $\eta(t) = \eta_0 \exp(-t / \tau_\eta)$, and the updates are applied for all the training patterns x over many epochs.

- The effect of each learning weight update is to move the weight vectors w_i of the winning neuron and its neighbours towards the input vector x. Repeated presentations of the training data thus leads to topological ordering.

Ordering and Convergence

- Provided the parameters (σ_0 , τ_σ , η_0 , τ_η) are selected properly, we can start from an initial state of complete disorder, and the SOM algorithm will gradually lead to an organized representation of activation patterns drawn from the input space. (However, it is possible to end up in a *metastable* state in which the feature map has a topological defect.)

- There are two identifiable phases of this adaptive process:

  - 1. Ordering or self-organizing phase – during which the topological ordering of the weight vectors takes place. Typically this will take as many as 1000 iterations of the SOM algorithm, and careful consideration needs to be given to the choice of neighbourhood and learning rate parameters.

  - 2. Convergence phase – during which the feature map is fine tuned and comes to provide an accurate statistical quantification of the input space. Typically the number of iterations in this phase will be at least 500 times the number of neurons in the network, and again the parameters must be chosen carefully.

Overview of the SOM Algorithm

- We have a spatially continuous input space, in which our input vectors live. The aim is to map from this to a low dimensional spatially discrete output space, the topology of which is formed by arranging a set of neurons in a grid. Our SOM provides such a nonlinear transformation called a feature map.

- The stages of the SOM algorithm can be summarized as follows:

  - 1. Initialization – Choose random values for the initial weight vectors wj .
  - 2. Sampling – Draw a sample training input vector x from the input space.
  - 3. Matching – Find the winning neuron I(x) with weight vector closest to input vector.
  - 4. Updating – Apply the weight update equation $\Delta w_{ji} = \eta(t)\, T_{j,I(\mathbf{x})}(t)\, (x_i - w_{ji})$.
  - 5. Continuation – keep returning to step 2 until the feature map stops changing.

Application: The Phonetic Typewriter

- One of the earliest and well known applications of the SOM is the phonetic typewriter of Kohonen. It is set in the field of speech recognition, and the problem is to classify phonemes in real time so that they could be used to drive a typewriter from dictation.

- The real speech signals obviously needed pre-processing before being applied to the SOM. A combination of filtering and Fourier transforming of data sampled every 9.83 ms from spoken words provided a set of 16 dimensional spectral vectors. These formed the input space of the SOM, and the output space was an 8 by 12 grid of nodes.

- Though the network was effectively trained on time-sliced speech waveforms, the output nodes became sensitized to phonemes and the relations between, because the network inputs were real speech signals which are naturally clustered around ideal phonemes. As a spoken word is processed, a path through output space maps out a phonetic transcription of the word. Some post-processing was required because phonemes are typically 40-400 ms long and span many time slices, but the system was surprisingly good at producing sensible strings of phonemes from real speech.